

专业硕士学位论文

基于代码语义向量表示和深度学习的软件 漏洞检测方法

Software Vulnerability Detection Method Based on Code Semantic Vector Representation and Deep Learning

张伟国

哈尔滨工业大学

2020 年 6 月

国内图书分类号：TP183
国际图书分类号：004.8

学校代码：10213
密级：公开

工程硕士学位论文

基于代码语义向量表示和深度学习的软件 漏洞检测方法

硕士研究生：张伟国

导师：苏小红教授

申请学位：工程硕士

学科：计算机技术

所在单位：计算机科学与技术学院

答辩日期：2020年6月

授予学位单位：哈尔滨工业大学

Classified Index: TP183

U.D.C: 004.8

Dissertation for the Master Degree in Science

**Software Vulnerability Detection Method Based on
Code Semantic Vector Representation and Deep
Learning**

Candidate:	Zhang Wei Guo
Supervisor:	Prof.Su Xiaohong
Academic Degree Applied for:	Master of Engineering
Speciality:	computer technology
Affiliation:	School of Computer Science and Technology
Date of Defence:	June, 2020
Degree-Confering-Institution:	Harbin Institute of Technology

摘要

近年来，随着互联网时代的不断发展和软件技术的不断提高，软件在人类的生活中发挥着越来越重要的作用，软件规模越来越大。同时，由于软件规模的不断扩大，开发人员的疏忽，导致软件的质量参差不齐，软件中存在的漏洞也越来越多，严重影响了软件的使用体验以及软件的安全性，对人类的生命财产安全产生了极大的威胁，越早检测出软件中存在的漏洞，损失就会越小，所以软件维护人员及时检测出软件中的漏洞就显得尤为重要。采用人工的方式检测代码中的漏洞复杂度比较高，同时也费时费力，所以要利用漏洞检测软件进行自动化的批量检测。

目前的代码漏洞检测方法，误报率和漏报率比较高，而且检测粒度比较粗，一般是文件级别的漏洞检测，所以本文的工作主要有两个部分：第一个主要部分是从代码中提取出特征，然后将特征进行表示学习，转化成向量，第二个主要部分是将特征向量输入到代码漏洞检测模型当中，通过代码漏洞检测模型输出代码有无漏洞的概率。关键和难点是要在代码中提取出有用的特征，能表现出代码漏洞的特征，然后将特征转化成能输入到代码漏洞检测模型当中的向量。这个向量化方式也很重要，可以根据代码语句的上下文信息学习到代码 `token` 之间的语义信息，另外针对不同的漏洞类型，还要采取不同的深度学习网络以获得相对较好的漏洞检测效果。

本文的主要工作如下：

首先对不同的漏洞数据集进行分析研究，选取质量较高的漏洞数据集。然后对漏洞数据集进行预处理，预处理首先生成源文件的抽象语法树和程序依赖图，然后根据抽象语法树中的节点和程序依赖图中的数据依赖和控制依赖关系进行程序切片，将切片转换成向量进行漏洞检测。本文对四种类型的漏洞进行检测，四种类型分别为指针使用相关的漏洞，数组使用相关的漏洞，算术表达式使用相关的漏洞，函数调用相关的漏洞，每一种漏洞的数据集单独进行实验。根据相应的关键点提取出对应类型的程序切片可以过滤掉一些无关语句，减少噪音，使预测结果更加准确。

其次，为了将提取出来的特征输入到漏洞检测模型当中，需要将其转换成向量，即向量化。首先利用 `word2vec` 得到词向量，然后在词向量的基础上进一步得到句子向量，最后，用这些漏洞代码的向量表示训练深度学习模型进行漏洞检测。

在基于深度学习的漏洞检测模型上，本文除了使用常用的 BiLSTM 模型

外，还使用了 TextCNN 以及 DesnseCNN 两种深度学习模型，并对这些模型的检测结果进行实验，此外还在模型中加入了注意力机制，进一步提高了模型对漏洞检测的准确率。

关键词：代码漏洞检测；程序切片；双向长短期记忆网络；卷积神经网络

Abstract

In recent years, with the continuous development of the Internet era and the continuous improvement of software technology, software plays an increasingly important role in human life, and the scale of software is growing. At the same time, due to the continuous expansion of software scale and negligence of developers, the quality of software varies, and there are more and more vulnerability in the software, which seriously affects the user's experience and security of the software, and poses a great threat to the safety of human life and property. The earlier the vulnerabilities in the software are detected, the smaller the loss will be, so it is very important for software maintainers to detect the vulnerabilities in software in time. It is very complex and time-consuming to detect vulnerabilities in code manually, so it is necessary to use vulnerability detection software for automatic batch detection.

The current code vulnerability detection methods have high underreporting rate and false alarm rate, and the detection granularity is relatively coarse, generally file level vulnerability detection, so the work of this paper mainly consists of two parts: the first part is to extract features from the code, and then to learn the representation of the features into vectors, the second part is to input the feature vector of the code slice in the vulnerability detection model, the probability of whether the output code is vulnerable through the code vulnerability detection model. The key and difficult point is to extract useful features in the code, which can show the characteristics of code vulnerabilities, and then transform the features into vectors that can be input into the code vulnerability detection model. This vectorization method is also very important. We need to learn the semantic information between code tokens according to the context information of code statements. In addition, we need to take different deep learning networks for different vulnerability types to achieve relatively good vulnerability detection results.

Summarize the main work of this paper:

First of all, different vulnerability data sets are analyzed, and high-quality vulnerability data sets are selected. Then preprocess the vulnerability data set. First, generate the abstract syntax tree and program dependency graph of the source file, and then slice the program according to the node in the abstract syntax tree and the data dependency and control dependency in the program dependency graph, and transform the slice into vector for vulnerability detection. In this paper,

four types of vulnerabilities are detected. They are Pointer use related vulnerabilities, array use related vulnerabilities, arithmetic expression use related vulnerabilities, function call related vulnerabilities. Each vulnerability's data set is tested separately. According to the corresponding key points, the corresponding type of program slice can filter out some irrelevant statements, reduce the noise, and make the prediction results more accurate.

Secondly, in order to input the extracted features into the vulnerability detection model, we need to transform them into vectors, that is, vectorization. First, word vector is obtained by word2vec, and then sentence vector is further obtained based on word vector. Finally, the vector representation of these vulnerability codes is used to train deep learning model for vulnerability detection.

Then for the extracted features, in order to input into the vulnerability detection model, it needs to be transformed into vector, that is, vectorization. First, word vector is obtained by word 2vec, then sentence vector is further obtained on the basis of word vector, and then machine learning method is used to preliminarily verify the effect. A better vectorization method is selected for subsequent training and testing of vulnerability detection model.

In the deep learning based vulnerability detection model, in addition to the commonly used bilstm model, this paper also uses two deep learning models, textcnn and desnsecnn, and tests the detection results of these models. In addition, attention mechanism is added to the model to further improve the accuracy of vulnerability detection.

Keywords: Code vulnerability detection; program slicing; bidirectional Longshort-term memory network; convolutional neural network

目 录

摘 要	II
ABSTRACT	IV
第 1 章 绪论	1
1.1 课题研究的背景和意义	1
1.2 国内外研究现状	2
1.2.1 代码表示和代码表示学习的国内外研究现状	3
1.2.2 漏洞检测方法的国内外研究现状	4
1.3 本文的主要研究内容以及各章节安排	5
第 2 章 基于程序切片的源代码中间表示方法	8
2.1 引言	8
2.2 相关概念和定义	8
2.2.1 程序依赖图的相关概念	9
2.2.2 程序切片的相关概念	9
2.3 数组使用相关漏洞的程序切片	10
2.3.1 切片关键点及切片过程	10
2.3.2 切片结果	12
2.4 算术表达式相关漏洞的程序切片	12
2.4.1 切片关键点及切片过程	13
2.4.2 切片结果	14
2.5 指针使用相关漏洞的程序切片	14
2.5.1 切片关键点及切片过程	14
2.5.2 切片结果	16
2.6 函数调用相关漏洞的程序切片	16
2.6.1 切片关键点及切片过程	17
2.6.2 切片结果	18
2.7 程序切片的结果	18
2.8 程序切片向量化	19
2.8.1 word2vec 的相关概念	20
2.8.2 word2vec 的训练	21

2.8.3 代码向量表示	21
2.9 本章小结	21
第 3 章 算法的实验结果验证	22
3.1 引言	22
3.2 机器学习实验结果	23
3.2.1 实验数据集	24
3.2.2 实验设置	24
3.2.3 评估指标	24
3.2.4 实验结果分析	25
3.3 基于 BiLSTM 的代码漏洞检测模型	26
3.4 基于 TextCNN 的代码漏洞检测模型	27
3.5 基于 DENSECNN 的代码漏洞检测模型	29
3.6 实验结果与分析	30
3.6.1 数据集信息	30
3.6.2 实验设置	31
3.6.3 评估指标	32
3.6.4 实验结果及分析	33
3.7 本章小结	44
结 论	46
参考文献	47
哈尔滨工业大学学位论文原创性声明和使用权限	52
致 谢	54

第1章 绪论

1.1 课题研究的背景和意义

随着大数据时代的到来和信息技术的不断发展，软件在人们的生活中也发挥着越来越重要的作用。

从日常的衣食住行到军事、科技等各个重要领域，软件已经无处不在。与此同时，软件的安全问题也日益突出。恶意攻击者常常利用软件中存在的安全漏洞对软件系统进行攻击，造成系统崩溃、信息泄露、财产损失等严重后果，对软件系统的安全造成了极大的威胁。因此，对软件安全漏洞的研究已经引起学术界和工业界的高度重视。

软件安全漏洞是指在软件设计与实现过程中由于算法的漏洞或者开发人员的疏忽，存在的一些可以被恶意攻击者所利用、对软件的安全使用构成威胁的缺陷或者不足。比较常见的软件安全漏洞包括指针解引用、缓冲区溢出、内存泄漏、整数溢出等。随着软件规模的不断增大和软件数量的不断增多，以及软件复杂度的不断提高，健壮的软件系统已经很难被设计和实现，所以软件漏洞的存在是不可避免的。因此，如何快速而准确地发现软件安全漏洞一直以来是信息安全领域的研究热点，因为越早发现软件中潜在的漏洞，就会减少由于软件漏洞带来的财产和安全损失。由此可见，软件安全漏洞检测对于保证软件系统安全具有十分重要的意义。软件安全漏洞往往来源于软件开发人员的疏忽，软件系统中潜在的不安全的代码，对代码的分析和挖掘一直以来是软件漏洞检测的重要手段，传统的软件漏洞检测主要依靠程序分析技术，如静态分析和动态分析，然而静态分析技术普遍存在误报率较高的缺点，而动态分析技术则容易产生漏报并且比较耗时。近几年来，深度学习又再次兴起，深度学习技术在许多领域取得了成功，随着开源代码量的增加，利用深度学习技术从大量的代码中学习软件漏洞模式也成为了可能，这为代码漏洞检测的研究提供了新的方向。

传统的漏洞检测方法需要安全专家进行人工定义规则，而修复人员可能不具备相关的知识。基于深度学习的漏洞检测方法不需要安全专家手工定义特征，可以从大量的历史数据中自动生成漏洞模式，有望改变软件源代码漏洞检测方法，使面向各种类型漏洞的漏洞模式从依赖专家手工定义向自动生成方向进行转变，并且显著提高漏洞检测的效果。然而，该方法的相关研究刚刚起步，大部分研究都集中在代码较粗粒度的漏洞检测上，比如说在函数级别或者文件级别上对代码漏洞进行检测，而对代码的“漏洞结构”的相关研究则不是很多。漏洞结构不仅能够使检

测工具判断代码是否包含漏洞，还能够指出漏洞在代码中存在的具体形式以及漏洞的产生位置，即定位到具体的代码片段。为了能让基于深度学习的漏洞检测方法能够在实际中更好的应用，对代码漏洞结构进行深入的研究就显得很有必要。

由于程序语言的特殊性质，导致漏洞并没有明显的细粒度代码结构来描述漏洞的候选区域，因此使用深度学习自动学出漏洞的代码结构比其他深度学习问题（比如目标检测^[1]）更加困难；以目标检测为例，对于给定的任意一幅图像，目标检测能够依据一定的算法和先验信息判断图像中是否存在指定类别的目标，如果存在，则返回目标的位置和类别的置信度。基于深度学习的代码漏洞检测过程类似于目标检测的过程，主要体现在代码漏洞检测也需要首先提取可能发生漏洞的相关语句，然后学习语句的向量表示，最后将得到的向量表示作为分类器的输入来判断代码是否存在漏洞。但是不同之处是图像中包含的对象有明显的区域结构，使用深度学习的目标检测方法能够把有明显区域结构的对象从背景中提取出来。但是代码中的漏洞语句没有明显的区域结构，和漏洞相关的语句在代码段中不一定是连续存在的，因此基于深度学习的方法很难提取没有明显区域结构的漏洞语句以达到漏洞检测的目的。

针对基于深度学习的代码漏洞特征提取方法无法学出漏洞细粒度结构的问题，本文使用深度学习模型展开对代码漏洞检测的研究。首先，从漏洞库中提取程序实例以及对应的标签，利用静态分析技术将程序实例转化为程序依赖图。然后以预定义的可能会引发缺陷的关键点为切片准则进行切片提取，并且将提取的切片加上相应的标签放入对应的数据库。如果切片中含有造成漏洞的语句，生成的程序切片加漏洞标签“1”，否则加漏洞标签“0”。最后将利用生成的针对不同关键点的数据作为数据集，进行切片级别的代码漏洞检测。

1.2 国内外研究现状

对代码进行漏洞检测也就是要进行一个分类任务，分类任务要提取出各种类别之间相区别的特征，以便取得更好的分类效果。在本文中对代码进行漏洞的检测，实际上就是对代码进行二分类任务，针对一段代码，有漏洞则为正样本，没有漏洞则为负样本。要找到相区别的特征就是要找到导致漏洞的特征，而这种特征刚好在无漏洞的代码中不存在。直接将源代码 token 化进行原函数级别的漏洞检测，粒度太粗，所以首先就是要对代码进行抽象化的表示，提取代码语句下更深层次更抽象的信息，以表现出代码中的语义，句法信息，因为这些信息可以体现出代码中包含的漏洞特征。代码的抽象化表示主要包括抽象语法树，程序依赖图，程序切片，API 调用序列等形式，通过将源代码进行抽象化表示，也能过滤掉一些漏洞无关的

语句,减少无关信息的干扰,在进行漏洞检测时,相当于一个去除噪音的过程。无论是机器学习模型还是深度学习模型,都需要将向量作为模型的输入,所以将代码进行抽象化表示之后,需要将得到的抽象语法树,API 调用序列等代码中间表示进行表示学习,即得到向量化的表示,并且向量化的表示需要学习到中间表示比如程序切片当中的语句之间的上下文信息。得到包含语义信息的向量后,将其输入到深度学习模型当中进行训练和预测。下面介绍代码表示与代码表示学习以及漏洞检测方法的国内外研究现状。

1.2.1 代码表示和代码表示学习的国内外研究现状

Guisheng Fan 等人^[2]在进行代码漏洞检测时采用的代码中间表示形式是抽象语法树(AST),首先利用 python 的工具包 javalang^[46]将源代码解析成抽象语法树,然后利用深度优先对抽象语法树进行遍历,抽象语法树中每一个节点都有一个类型,遍历的过程中判断当前遍历的节点的类型,如果属于预定义的节点类型则保留该节点,遍历结束后,将保留的节点按照出现频率由大到小排序,依次映射成数字 1,2,3...,并固定这个向量大小为 n ,超过 n 的部分,先删除大的数字,因为大的数字对应的节点出现的频率更小。不足 n 的部分,进行补 0。然后利用嵌入层进行表示学习,将上一步映射的数字转换成高维实数向量。

Li 等人^[3]提出了 code gadget 的概念,简单来说,就是从源代码中提取出的一些和漏洞相关的代码片段。首先将源代码转化为抽象语法树,然后遍历抽象语法树,判断其中每一个节点的节点类型,该文章主要关注的是库函数和 API 函数调用相关的漏洞,所以如果节点类型属于库函数调用类型或者是 API 函数调用,则提取出对应调用关系中包含的参数,然后根据参数和对应的函数进行切片,最后利用 word2vec 将切片中的 token 进行向量化。

Y Li 等人^[4]利用 API 调用序列作为代码的中间表示形式来进行漏洞检测,首先对 API 调用序列和开源软件的漏洞之间的关系进行研究,根据研究结果建立一个 API 函数库。作者通过研究 CWE^[5],NVD^[6],CNNVD^[7],对比这几种漏洞分类方法,将漏洞分为和十种操作类型相关。通过研究这十种操作类型相关的漏洞代码,得到每种操作相关漏洞的关键 API,然后用开源软件工具 Cuckoo Sandbox 运行程序,获得 API 的调用序列。然后将 API 调用序列向量化。向量化后一个 20*20 的二维矩阵,具体过程是:共十种类型,每种类型对应 2 行,40 个元素。每个元素对应于该类型的一个 API 函数,如果代码中存在,则该元素为对应 API 函数调用次数,如果没有被调用则该元素为 0,如果该种类型的 API 函数不足 40 个,则对应位置的元素为-1。

Li 等人^[8]采用的代码中间表示形式是程序切片，将代码根据不同的预定义关键点进行前向和后向切片，这样可以过滤掉一些漏洞无关语句，降低噪音的影响。该文章根据漏洞检测工具 checkmarx^[9]的规则，将漏洞分类为四大类，分别是函数调用相关的漏洞，数组使用相关的漏洞，算术表达式使用相关的漏洞以及指针使用相关的漏洞。通过遍历抽象语法树，判断节点类型，匹配关键点，匹配成功则生成切片标准，根据切片标准生成对应类型的切片。采用的向量化方法是利用 word2vec^[10]将得到的切片中的每一行语句进行向量化，对于每一条语句当中的每个词，利用 word2vec 训练得到词向量，然后将词向量进行拼接得到句子向量。

Gu 等人^[11]没有进行代码漏洞检测任务，但是在利用深度学习的代码搜索中也用到了代码的中间表示，将源代码根据空格分割成 token 序列，作为第一种中间表示。利用 JDT 生成对应方法的抽象语法树，然后通过遍历对应方法的抽象语法树生成 API 调用序列作为第二种中间表示，最后将函数的名字第三种中间表示，然后利用 word2vec 将得到的所有中间表示训练为词向量。

Russell 等人^[12]在做代码漏洞检测时，利用的是源代码的 token 化形式，将源代码中的每一行的每一个 token 作为中间表示，然后利用 word2vec 向量化。每一个 token 对应的向量作为一行，后续进行卷积操作。

Sampaio 等人^[13]根据对源代码的上下文敏感数据流分析获得代码的中间表示，进行代码漏洞检测。Shin 等人^[14]将代码度量作为特征来进行漏洞检测，主要包括代码行数，方法个数，圈复杂度等等，将这些代码度量作为漏洞检测的特征。Nguyen 等人^[15]根据代码的依赖图获得代码片段，然后利用嵌入层转化成向量。R 等人^[16]利用文本挖掘技术进行漏洞检测，将每一个 token 和 token 出现的次数作为特征，转化成向量对漏洞进行预测。Dam 等人^[43]将代码根据函数分割，每一个函数都分别 token 化，变成 token 序列，每个序列分别利用 LSTM 提取深层次的特征。Li 等人^[6]后来提出了新的改进方法，利用 LLVM 编译器得到中间代码，加上程序切片作为代码的中间表示，进一步利用 word2vec 转化为向量用于训练和预测深度学习网络。

大多数的代码漏洞检测均采用 word2vec 作为特征向量化的方法，本文也采用 word2vec 将程序切片进行词嵌入，得到向量。

1.2.2 漏洞检测方法的国内外研究现状

对得到的代码中间表示以及向量化的表示需要用深度学习模型进行分类，目前有很多模型已经应用于漏洞检测。

Guisheng Fan 等人^[2]将得到的向量利用单层感知机与注意力机制进行结合，对

漏洞进行检测, 将特征向量 F 输入全连接层, 利用 softmax 得到注意力分数 S , 再与特征向量 F 点乘得到加权后的特征向量 SF 。然后分别利用卷积神经网络(CNN)和递归神经网络(RNN)进行训练和预测, 这个随机初始化的向量 A 是可以在训练过程中不断更新优化的。在不同的项目上, CNN 和 RNN 有不同的表现。Li^[3]等人通过分析漏洞关键点, 将函数进行切片, 做切片级别的漏洞检测, 将切片利用 word2vec 向量化, 将得到的特征向量利用双向长短期记忆网络(BiLSTM)连接 softmax 进行训练和预测。Y Li 等人^[4]利用程序分析工具得到抽象语法树, 遍历抽象语法树得到 API 调用序列, 然后同样利用 word2vec 转换成向量, 将得到的特征向量利用 CNN 和 RNN 结合进行训练和预测, 首先是用 CNN 对二维矩阵特征向量进行卷积, 池化, 然后连接 RNN 网络, CNN 学习得到一部分上下文特征, RNN 进一步学习得到代码的上下文语义信息。Li 等人^[4]将根据关键点获得切片标准, 进一步获得程序切片, 利用 word2vec 将切片转换成向量, 然后利用 GRU, LSTM, BiLSTM, BiGRU, CNN 分别进行实验, 最终在不同的评价指标上 CNN, BiLSTM 和 BiGRU 的效果最好。Russell 等人^[12]是将源代码直接分解成一个个 token, 然后将 token 训练成词向量, 每一个 token 对应的词向量作为一行, 一个函数就构成了一个二维向量矩阵, 这个二维向量矩阵作为 CNN 的输入, 进行卷积, 最大池化, 将不同卷积核最大池化的结果进行拼接, 得到一维向量, 然后利用随机森林进行分类。

目前主要应用于漏洞检测的模型中, 机器学习模型主要应用到了随机森林, 支持向量机(SVM), 深度学习模型主要应用到了 BiLSTM, CNN 以及注意力机制。漏洞检测粒度最细的是 Li^[3]等人对程序切片进行的漏洞检测。判断粒度较细的切片是否有漏洞, 更有助于修复人员对漏洞代码进行定位。

1.3 本文的主要研究内容以及各章节安排

本文主要内容分为两大部分, 第一部分内容是对代码的中间表示程序切片进行研究, 实验。通过四种漏洞相关的关键点对源代码的抽象语法树进行遍历, 匹配关键点, 然后根据关键点生成切片标准, 利用切片标准生成对应的程序切片然后需要根据源文件对切片进行打标签, 包含漏洞语句的切片标记为 1, 不包含漏洞语句的切片标记为 0, 然后对四种切片进行表示学习, 得到切片的向量表示, 然后利用机器学习方法支持向量机 (SVM), 随机森林 (RF), LGBM 快速验证其有效性。第二部分内容是对代码漏洞检测方法的研究, 实验。前面得到了四种类型的漏洞切片, 对每种类型漏洞的切片得到的向量, 分别应用深度学习网络 BiLSTM, TextCNN 以及 DenseCNN 进行训练和预测并应用注意力机制, 对比其评估指标, 找到每种

漏洞相对比较合适的漏洞检测方法，并对比有无注意力机制的漏洞检测效果有何不同。

本文将 C/C++源程序进行切片，应用深度学习网络 BiLSTM，TextCNN 和 DenseCNN 对代码漏洞进行检测，比源文件级别和函数级别的漏洞检测粒度更细，主要流程如图 1-1。

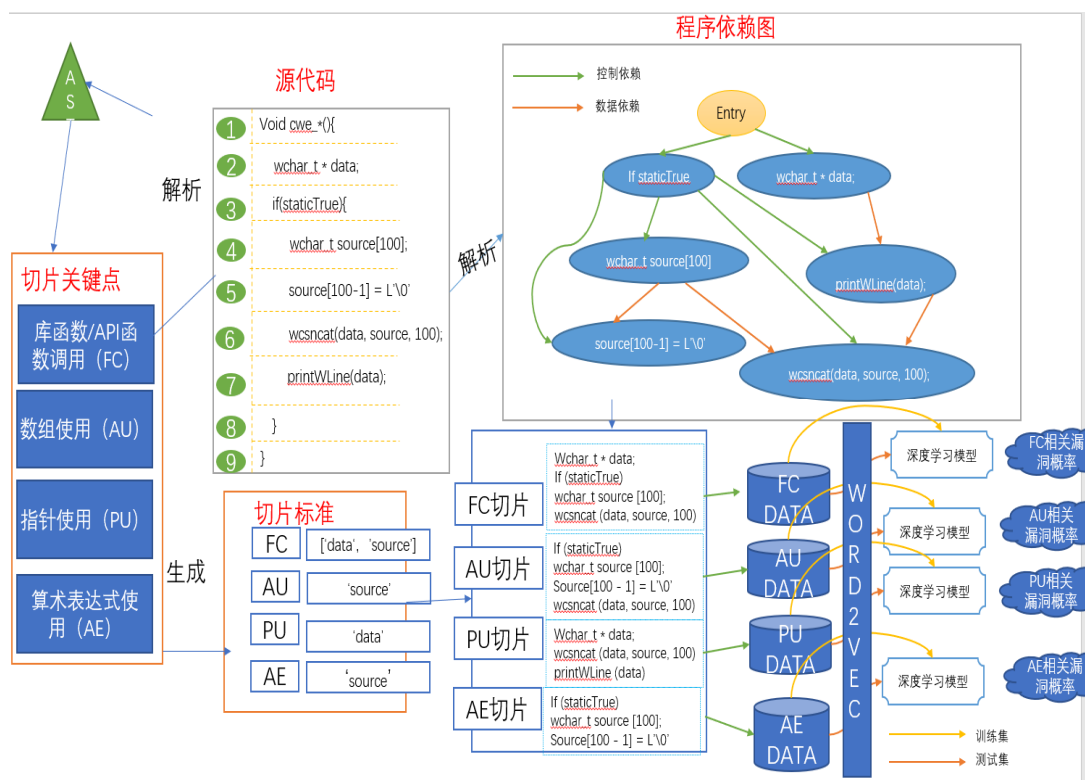


图 1-1 本文研究内容主要流程

本文的其它章节结构以及其主要内容如下所示：

第 2 章介绍了程序切片的主要概念以及相关的程序依赖图（PDG）的主要概念。然后介绍了本文进行漏洞检测时主要用到的四种类型的切片，数组使用相关的切片关键点以及其切片过程，算术表达式相关的切片关键点以及其切片过程，指针使用相关的切片关键点以及其切片过程，函数调用相关的切片关键点以及其切片过程。并给出每一种切片的具体例子，对切片的具体过程进行说明，最后对程序切片的实验结果进行分析。

第 3 章介绍了程序切片的表示学习方法，首先介绍 word2vec 的相关概念以及训练过程，简要介绍了 skip-gram 模型和 CBOW 模型，并说明了两个模型的不同，然后针对四种漏洞类型的切片的数据集分别训练词向量。最后用机器学习方法进行实验验证，对数据集，评估指标进行介绍，对实验结果进行分析，说明 word2vec 这种表示学习方法在做漏洞检测时的有效性。

第 4 章介绍了深度学习网络当中的卷积神经网络模型（CNN），双向长短期记忆网络模型（BiLSTM）以及 CNN 网络模型和其应用于文本分类的改进模型 DenseCNN^[17]。根据切片标准得到切片，经 word2vec 表示学习之后向量化，分割成训练集，验证集以及测试集。然后将网络配置好，用分割好的数据集进行实验，对比分析不同的深度学习网络对四种类型的漏洞的预测效果，然后对比有无注意力机制对四种漏洞检测效果的影响。

最后，对整篇文章进行总结，分析用到的漏洞检测方法的优点以及尚存在的不足之处，最后对未来需要进行的工作进行总结，展望。

第 2 章 基于程序切片的源代码中间表示方法

2.1 引言

对于程序源代码，里面包含的信息有很多，其中最直观的信息主要包括代码和注释。另外由于不同的开发人员也具有不同的编码方式，比如命名规则，代码风格等等，所以直接在源代码上进行漏洞检测，会受到一些和漏洞不相关的信息的干扰，所以需要对这些干扰信息进行过滤以减少噪音，要把源代码转换成中间表示的形式，比如抽象语法树(AST)，程序依赖图(PDG)，程序切片，函数调用序列等等，然后从中间表示上获取信息，再进一步将获取到的信息进行表示学习，转换成向量的形式。

利用现有的 C++ 代码分析工具 Joern^[18]可以获取到 C++ 代码的抽象语法树，程序依赖图。通过结合这两种抽象表示，获取到可能和漏洞相关的关键点的切片。本文关注四类漏洞，分别是数组使用相关的漏洞，算术表达式相关的漏洞，函数调用相关的漏洞，指针使用相关的漏洞。根据相应的关键点，遍历抽象语法树的每一个节点，抽象语法树中的每一个节点都属于一种类型，如果当前节点的类型匹配了预定义的四类漏洞关键点的类型，则利用该节点对应代码以及对应代码中的参数作为切片标准，对源代码进行切片。

本文关注四类漏洞，所以进行四种类型的切片，数据集采用的是 MVD^[13]对应的源代码，源代码是文件级别的，本文对函数进行程序切片。首先配置 C++ 代码分析工具 Joern，将分析结果存储在图数据库 Neo4j 中，然后利用 Joern 生成源代码的抽象语法树，生成对应源代码的程序依赖图(PDG)，根据抽象语法树的节点类型和程序依赖图体现的数据依赖关系和控制依赖关系对源代码进行切片，然后将切片和源代码进行对比，如果包含漏洞语句，则切片的标签为 1，如果不包含漏洞语句，则切片的标签为 0。

2.2 相关概念和定义

在做代码漏洞检测之前，首先要对源代码进行切片，切片是源代码的一些语句的组合。利用四类漏洞关键点对源代码进行切片，提取出感兴趣的部分，相当于过滤掉了一些和漏洞无关的语句，减少噪音的干扰，提高漏洞检测效果。首先本节对程序切片涉及到的相关概念和定义做一些阐述说明，然后介绍程序切片算法，然后说明本文关注的四类漏洞的切片的关键点以及切片过程，最后对切片结果进行分析。

2.2.1 程序依赖图的相关概念

程序依赖图是由 K.J.Ottenstein^[19] 和 L.M.Ottenstein^[20]两位学者在 1984 年提出的, 该图体现了源代码中的依赖关系, 包括数据依赖关系和控制依赖关系。首先阐述下控制流图^[21] (CFG) 的概念, 然后说明数据依赖和控制依赖的含义, 最后介绍程序依赖图的概念。

控制流图由顶点集和边集组成, 源文件中的每一个函数都对应一个控制流图, 其中顶点集中的每一个顶点都代表源代码中的一条语句或者是一个控制谓词。其中边集中的边是有向边, 每一条有向边代表图中两个节点之间存在的控制流关系。

数据依赖是指在一个函数对应的控制流图中, 如果在节点 B 处用到了在节点 A 处计算的值, 则节点 B 数据依赖于节点 A。控制依赖是指在一个函数对应的控制流图中有节点 A 和节点 B, 如果节点 A 到程序的结尾的所有路径, 都必须通过节点 B, 则称节点 B 后支配节点 A。如果在节点 C 和节点 D 之间存在一条路径, 并且开始于 C, 结束于 D, 并且节点 D 后支配路径上的每一个节点, 但是节点 D 不后支配节点 C, 则称节点 D 控制依赖于节点 C。

在数据依赖和控制依赖的基础上, 说明程序依赖图的定义。程序依赖图也是由顶点和有向边组成的, 和控制流图唯一不同的是, 其中任意两个节点间的有向边代表节点间存在的数据依赖或者控制依赖关系。

2.2.2 程序切片的相关概念

程序切片^{[22][23]}是源代码中的一些语句的组合, 切片中代码的顺序和源代码中的顺序一致, 具体来说是指从源代码中提取出的和关键点相关的语句, 和关键点无关的语句全部过滤掉。针对切片准则来说, 获得的切片和源代码在语义上是一致的。在本文中, 要对软件的漏洞进行检测, 将软件漏洞分为四种类型, 所以根据四种漏洞类型的关键点对源代码进行切片。程序切片分为前向切片和后向切片, 前向切片是指在程序中受关键点影响的语句, 这种语句出现在关键点的后面。后向切片是指在程序中对关键点造成影响的语句, 即出现在关键点前面并且和关键点之间存在数据依赖或者控制依赖的语句。将前向切片和后向切片结合即得到完整的程序切片^[24]。

获得程序切片的算法依次出现了依据控制流图的数据流方程算法, 基于 PDG 的图可达算法, 以及基于 SDG^[25] (系统依赖图) 的图可达算法。数据流方程方法需要对很多的方程进行求解, 而且这些方程是迭代形式的, 所以求

解的过程比较复杂，求解需要过多的时间和空间，对于过于复杂的程序无法获得程序切片。基于 PDG 的图可达的切片算法使切片的过程变的简单并且可视化，而且可以获得包含过程的函数的切片，但不能解决包含多个过程的函数的切片问题。后期出现的基于 SDG 的图可达算法可以解决程序中包含多个过程的切片问题。利用 C++ 代码分析工具 Joern 可以直接解析 C++ 源代码获得其程序依赖图（PDG），相对于其它方法比较方便，所以本文采用的是基于 PDG 的图可达算法^[8]对源程序进行切片。首先针对每一个函数的源代码生成抽象语法树，根据语法树和漏洞关键点生成切片准则，进一步根据切片准则生成程序的切片。后续使用的切片示例程序如图 2-1 所示。

```
1 void cwe_*(){
2     wchar_t * data;
3     if(staticTrue) {
4         wchar_t source[100];
5         source[100-1] = '\\0'; /* null terminate */
6         /* FLAW: (EXC_BAD_ACCESS) Variable 'data'
7         is uninitialized when used here */
8         wcsncat(data, source, 100);
9         printWLine(data);
10    }
11 }
```

图 2-1 切片示例源程序

2.3 数组使用相关漏洞的程序切片

数组是有序的元素序列，它可以存储一个固定大小的相同类型元素的顺序集合。对数组的不恰当使用，包括不合法的数组元素访问或者不恰当的数组运算造成的代码漏洞，称为由“数组的使用”造成的代码漏洞，其中数组的使用为造成漏洞的关键点。根据此关键点对代码进行切片，过滤出源代码中只和数组的使用相关的语句，作为漏洞检测的数据，总体过程包括以下两个步骤：（1）根据关键点生成切片标准。（2）然后根据切片标准，生成程序切片。

2.3.1 切片关键点及切片过程

首先将源程序解析成抽象语法树，然后遍历抽象语法树，查找抽象语法

树中节点类型为标识符定义语句的中间节点（即 IdentifierDeclStatement），然后判断查找出的中间节点所对应的语句中是否含有符号 “[” 或者 “]”，如果这些条件都满足，则将此标识符定义语句作为切片标准的语句 S，并且提取语句中定义的参数变量作为切片标准的 V。最后将 S 和 V 组合成最终的切片标准（S, V）。

对于示例程序，如图 2-2 在其对应的抽象语法树(AST)中匹配到关键点，根据数组使用作为关键点提取的切片标准为（wchar_t source[100]; , [source]）。

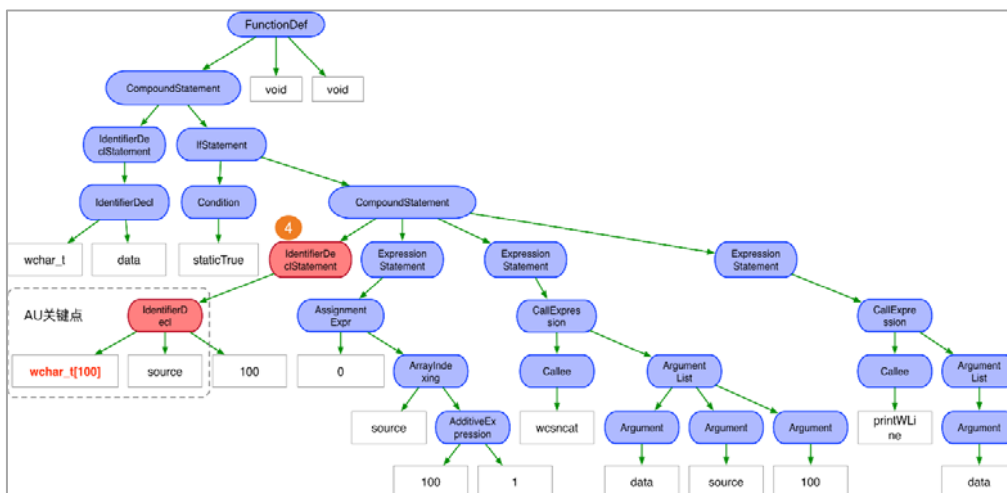


图 2-2 匹配数组使用关键点

通过分析上述示例，可知在根据“数组的使用”关键点生成切片标准(S, V)的过程中语句 S 需要满足以下两个条件：（1）语句中包含标识符定义语句（IdentifierDeclStatement）；（2）定义语句对应的代码中包含符号 “[” 或者 “]”，因为数组定义和使用时都会用到 “[” 和 “]”。

然后利用上面的切片标准（S, V）对源程序进行切片，判断程序依赖图（PDG）中和切片标准（S, V）存在数据依赖和控制依赖关系的语句，分别构成前向切片和后向切片，组合成最终的切片结果。针对图 2-1 的示例程序，其对应的程序依赖图以及切片分析过程如图 2-3。切片标准为（wchar_t source[100]; , [source]），图中绿色的线代表控制依赖，红色的线代表可达关系，语句“if(staticTrue)”和切片标准“（wchar_t source[100]; , [source]）”之间存在控制依赖，并且“if(staticTrue)”出现在切片标准的前面，对切片标准对应的有影响，所以将其作为针对切片标准的后向切片的一部分。语句“Source[100-1]=L'\0';”和切片标准“（wchar_t source[100]; , [source]）”之间存在控制依赖，语句“wcsncat(data, source, 100)”和切片标准之间

也存在控制依赖，并且这两条语句都出现在切片标准的后面，都会受到切片标准对应语句的影响，所以将其作为针对切片标准的前向切片的一部分。

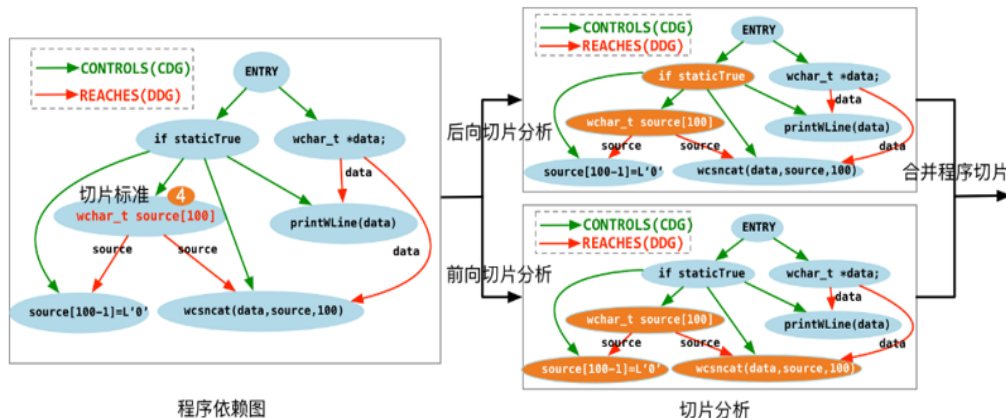


图 2-3 数组使用相关切片过程

2.3.2 切片结果

根据 source 和 data 在程序依赖图当中和别的节点之间的数据依赖关系和控制依赖关系生成以数组使用为关键点的切片。切片结果如图 2-4。

3	if(staticTrue)
4	wchar_t source[100];
5	source[100-1] = '\0';
6	wcsncat(data, source, 100);

图 2-4 数组使用为关键点的切片

2.4 算术表达式相关漏洞的程序切片

算术表达式是指用算术运算符和括号将运算对象（也称操作数）连接起来的、符合程序语法规则的式子。由于算术表达式的不恰当使用所引发的代码漏洞称为以算术表达式为关键点的代码漏洞（例如：整型溢出）。同样以图 2-3 中的代码段为例，分析算术表达式关键点的提取以及根据关键点生成程序切片的过程。其中算术表达式的使用为造成漏洞的关键点。根据此关键点对代码进行切片，过滤出源代码中只和算术表达式的使用相关的语句，作为漏洞检测的数据，步骤包括：（1）根据关键点生成切片标准。（2）根据切片标准，生成程序切片。

2.4.1 切片关键点及切片过程

首先，将代码段解析成语法树形式，其次在语法树中查找类型为 `ExpressionStatement` 的中间节点，并且判断 `ExpressionStatement` 节点所对应的代码语句中是否包含符号“=”，以及符号“=”右侧的标识符是否大于等于 1 个。如果满足上述条件，则 `ExpressionStatement` 节点对应的语句即为切片标准中的 `S`。然后提取语句中的标识符作为切片标准的 `V`。最后，利用得到的切片标准 (`S,V`) 用于提取切片。

对于示例程序，如图 2-5 在其对应的抽象语法树 (AST) 中匹配到关键点，根据“算术表达式”为关键点提取的切片标准为 (`source[100-1] = '\0';`, `[source]`)。

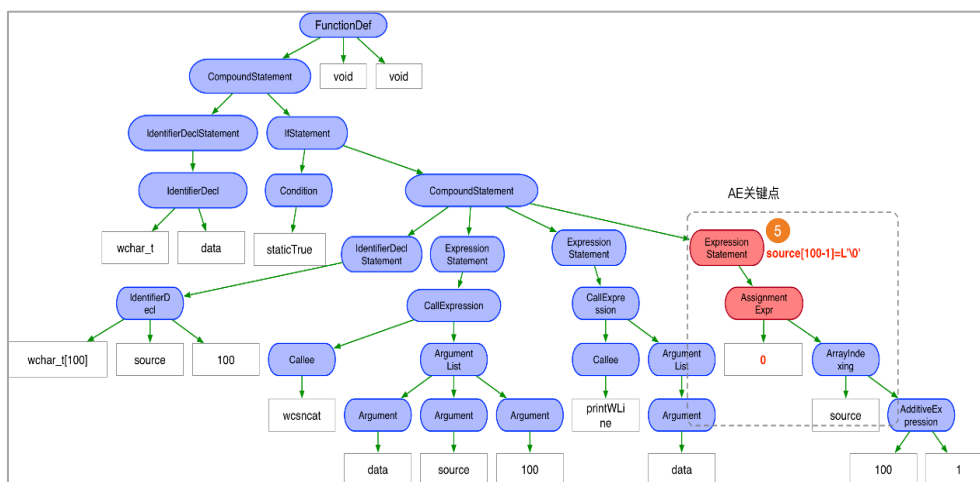


图 2-5 匹配算术表达式关键点

通过分析上述示例，可知在根据“算术表达式”的关键点生成切片标准 (`S, V`) 的过程中语句 `S` 需要满足以下两个条件：（1）语句中包含表达式语句 (`ExpressionStatement`)；（2）定义语句对应的代码中包含符号“=”，并且“=”右侧的标识符数量大于等于 1。

然后利用上面的切片标准 (`S, V`) 对源程序进行切片，判断程序依赖图 (`PDG`) 中和切片标准 (`S, V`) 存在数据依赖和控制依赖关系的语句，分别构成前向切片和后向切片，组合成最终的切片结果。针对图 2-1 的示例程序，其对应的程序依赖图以及切片分析过程如图 2-6。切片标准为 (`source[100-1] = '\0';`, `[source]`)，语句“`wchar_t_source[100]`”和切片标准之间存在可达关系。语句“`if(staticTrue)`”和切片标准之间存在控制依赖关系，并且这两条语句均出现在切片标准的前面，对切片标准产生影响，所以将这两条语句作为针对切片标准的后向切片的一部分。切片标准在程序依赖图中处于末尾位置，

所以不包含前向切片。

←

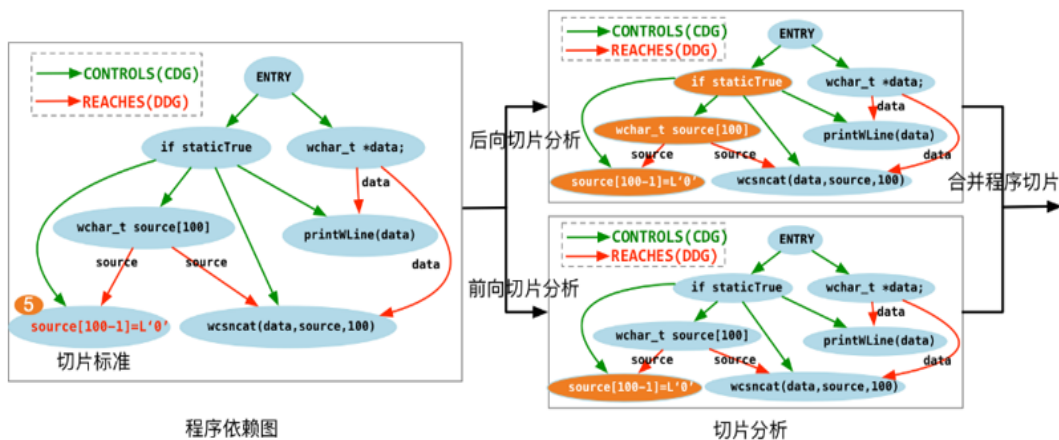


图 2-6 算术表达式相关切片过程

2.4.2 切片结果

根据关键点与切片标准在程序依赖图当中和别的节点之间的数据依赖关系和控制依赖关系生成以算术表达式为关键点的切片，切片结果如图 2-7。

3	if(staticTrue)
4	wchar_t source[100];
5	source[100-1] = '\0';

图 2-7 算术表达式为关键点的切片

2.5 指针使用相关漏洞的程序切片

指针是一个变量，其值为另一个变量的地址，即，内存位置的直接地址。对指针的不恰当使用，赋值或访问造成的代码漏洞，称为由“指针的使用”造成的代码漏洞，其中定义指针的语句为造成漏洞的关键点。仍然以表 1 中的代码段为示例，分析“指针的使用”关键点的提取和以此为关键点生成程序切片的过程。其中指针的使用为造成漏洞的关键点。根据此关键点对代码进行切片，过滤出源代码中只和指针的使用相关的语句，作为漏洞检测的数据。整体过程分为以下两个部分：（1）根据关键点生成切片标准。（2）根据切片标准，生成程序切片。

2.5.1 切片关键点及切片过程

首先，将源程序解析成抽象语法树形式，遍历抽象语法树，然后查找语

法树中标识符定义的中间节点，如果标识符定义的中间节点对应的代码语句包括符号“*”，则将此语句作为程序切片标准的语句 S。最后提取语句 S 中定义的指针作为程序切片标准中的标识符 V，并且将得到的 S 和 V 组合成最终用于提取切片的切片标准 (S,V)。

对于示例程序，如图 2-8 在其对应的抽象语法树(AST)中匹配到关键点，根据“指针的使用”为关键点提取的切片标准为 (wchar_t * data; , [data])。

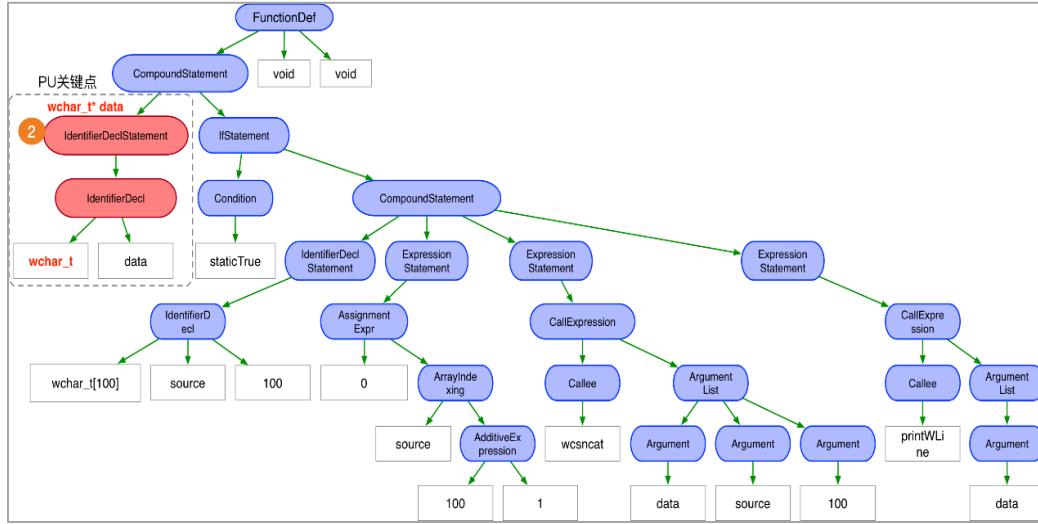


图 2-8 匹配指针使用关键点

通过分析上述示例，可知在根据“指针的使用”关键点生成切片标准(S,V)的过程中语句 S 需要满足以下两个条件：（1）语句中包含标识符定义语句（IdentifierDeclStatement）；（2）定义语句对应的代码中包含符号“*”。

然后利用上面的切片标准 (S, V) 对源程序进行切片，判断程序依赖图 (PDG) 中和切片标准 (S, V) 存在数据依赖和控制依赖关系的语句，分别构成前向切片和后向切片，组合成最终的切片结果。针对图 2-3 的示例程序，其对应的程序依赖图以及切片分析过程如图 2-9。切片标准为 (wchar_t * data; , [data])，在对其进行前向切片分析时，发现切片标准处于程序依赖图的入口，所以不包含前向切片。语句 “wcsncat(data, source, 100)” 和切片标准之间存在可达关系，语句 “printWLine (data)” 和切片标准之间也存在可达关系，并且这两个语句都出现在切片标准的后面，受到切片标准的影响，所以将其作为针对切片标准的前向切片的一部分。

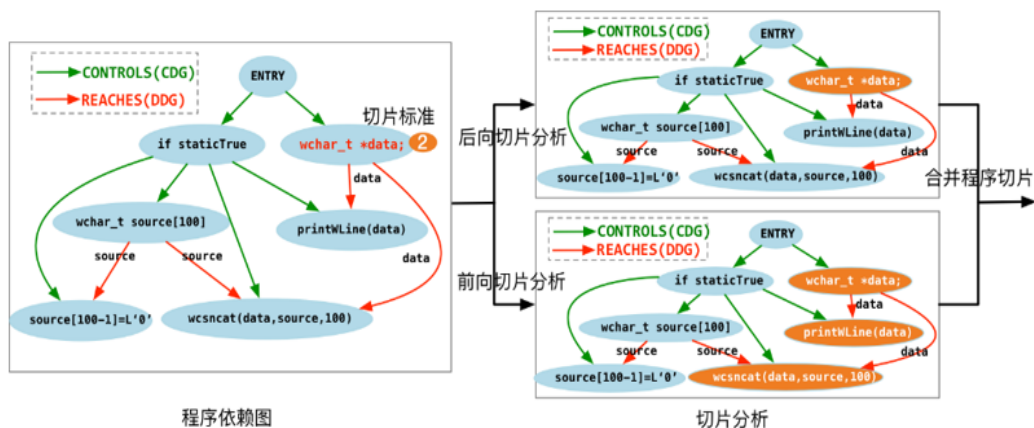


图 2-9 指针使用相关切片过程

2.5.2 切片结果

根据关键点与切片标准在程序依赖图当中和别的节点之间的数据依赖关系和控制依赖关系生成以指针使用为关键点的切片，切片结果如图 2-10。

2	wchar_t * data;
6	wcsncat(data, source, 100);
7	printWLine(data);

图 2-10 指针使用为关键点的切片

2.6 函数调用相关漏洞的程序切片

库函数是指编译器提供的可在 C 源程序中调用的函数。可分为两类，一类是 C 语言标准规定的库函数，一类是编译器特定的库函数。如果某代码漏洞是由库函数调用引起的，则程序中的库函数即为造成此代码漏洞的关键点。比如说示例程序中的代码漏洞发生在语句“wcsncat(data, source, 100);”，并且语句中执行了调用库函数“wcsncat”的操作，因此此漏洞可以称为由库函数调用产生的代码漏洞。本方案主要考虑了 811 个可能引发漏洞的库函数。本文以表 2-1 中的代码段为示例分析以库函数的调用为关键点的程序切片的提取过程。其中函数的调用为造成漏洞的关键点。根据此关键点对代码进行切片，过滤出源代码中只和函数调用相关的语句，作为漏洞检测的数据，步骤包括：（1）根据关键点生成切片标准。（2）根据切片标准，生成程序切片。

2.6.1 切片关键点及切片过程

首先，利用程序分析技术将示例程序解析成抽象语法树的形式，然后对抽象语法树进行遍历，通过节点匹配查找类型为“Callee”的树中的中间节点，如果“Callee”节点所对应的标识符（identify）在预先定义的可能引发漏洞的库函数列表中，则“Callee”节点所对应的父节点（CallExpression），即为切片标准中的语句 S。最后从语句 S 中提取涉及到的变量作为切片标准中的 V，结合 S 和 V 最终得到用于提取切片的切片标准（S,V）。

对于示例程序，如图 2-11 在其对应的抽象语法树（AST）中匹配到关键点，根据库函数调用为关键点提取的切片标准为（wcsncat(data,source,100); , [data,source]）。

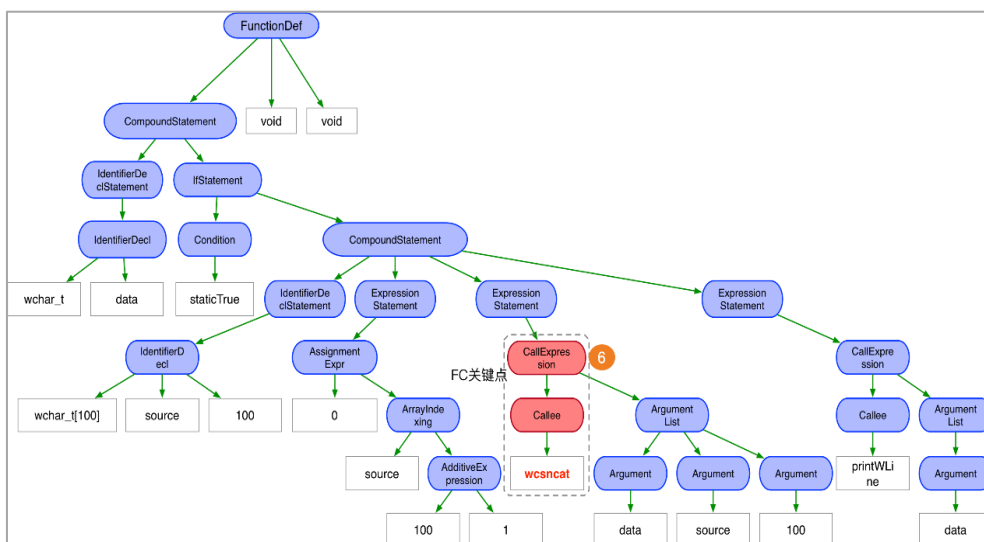


图 2-11 匹配函数调用关键点

通过分析上述示例，可知在根据“库函数调用”关键点生成切片标准(S,V)的过程中语句 S 需要满足以下两个条件：（1）语句中包含库函数调用（callee）；（2）库函数名在预定义的函数列表中。

然后利用上面的切片标准（S，V）对源程序进行切片，判断程序依赖图（PDG）中和切片标准（S，V）存在数据依赖和控制依赖关系的语句，分别构成前向切片和后向切片，组合成最终的切片结果。针对图 2-1 的示例程序，其对应的程序依赖图以及切片分析过程如图 2-12，切片标准为（wcsncat(data,source,100); , [data,source]），语句“if(staticTrue)”和切片标准之间存在控制依赖，语句“wchar_t source[100]”和切片标准之间存在数据依赖关系，语句“wchar_t * data;”和切片标准之间也存在数据依赖关系，并且这三条语句

都出现在切片标准之前，对切片标准造成影响，所以将其作为后向切片的一部分。对切片标准进行前向切片分析时，发现切片标准处于程序依赖图的末尾，所以其不包含前向切片。

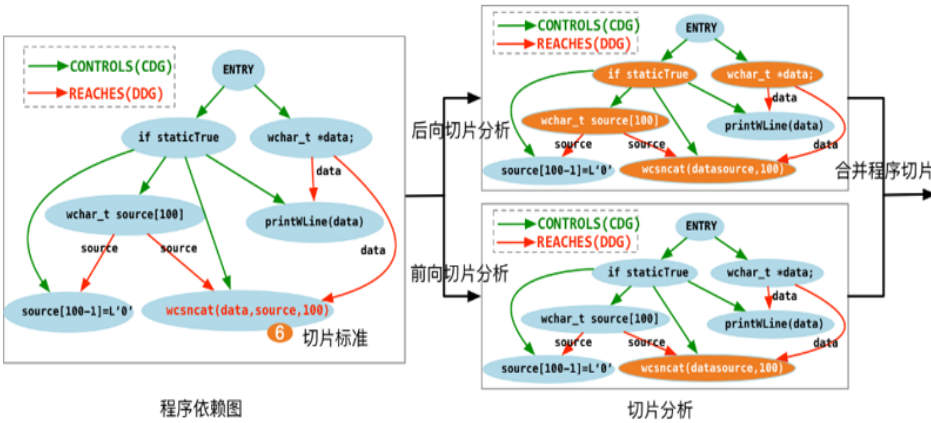


图 2-12 函数调用相关切片过程

2.6.2 切片结果

根据关键点与切片标准在程序依赖图当中和别的节点之间的数据依赖关系和控制依赖关系生成以函数调用为关键点的切片，切片结果如图 2-13。

2	wchar_t * data;
3	if(staticTrue) {
4	wchar_t source[100];
6	wcsncat(data, source, 100);

图 2-13 函数调用为关键点的切片

2.7 程序切片的结果

本节主要说明所使用的源代码和对源代码进行切片的统计结果，这部分得到的切片将作为后文的漏洞检测数据集。本文采用的数据集是 L 等人[8]使用的 MVD 数据集的源代码，使用了其中带有 CWE 标签的数据，因为其中的函数用 bad 和 good 命名，所以打标签比较方便。利用 joern 解析源代码生成抽象语法树和程序依赖图，并将图信息存储在图数据库 Neo4j 中，通过图查询语句，遍历抽象语法树，按照上文给出的切片方法得到四种漏洞相关的切片，并对切片打标签。good 函数切出来的切片直接将标签标记为 0，bad 函数切出来的切片需要和原函数进行对比，在原函数中，漏洞语句所在代码行有

标记，如果切片中包含有漏洞的语句，则该切片标记为 1，不包含有漏洞的语句，则该切片标记为 0。具体来讲就是，原函数中每一行导致漏洞的代码上方都会有一条注释。通过判断切片中每一行语句在原函数中的位置对切片进行打标签，如果当前判断的语句在原函数中上方有‘FLAW’注释，则切片打标签为 1，否则打标签为 0。四种漏洞相关的切片数量统计如表 2-1 所示。

表 2-1 漏洞数据类型及数量

漏洞数据类型	数量
数组相关有漏洞切片	1677
数组相关无漏洞切片	10760
算术表达式相关有漏洞切片	7210
算术表达式相关无漏洞切片	53082
指针使用相关有漏洞切片	2676
指针使用相关无漏洞切片	10779
函数调用相关有漏洞切片	2985
函数调用相关无漏洞切片	14016

2.8 程序切片向量化

在上一章中介绍了基于程序切片的源代码中间表示方法，将源代码根据四种漏洞相关的关键点，得到切片标准，进行切片。得到了带有标签的切片后，需要将其转换成向量，即进行切片的表示学习。早期的向量化方法会用到独热向量（One-hot embedding），但是这种方法得到的向量过于稀疏，容易导致维度爆炸。目前常用的表示学习方法有 word2vec，fasttext^{[26][27][28]}，glove^[29]。

Word2vec 分为两种模型，skip-gram 模型和连续词袋模型（CBOW），前者是通过一个词来预测周围的词，使周围的词在输出层的概率最大来训练网络。后者是根据周围的词来预测中心词，是中心词在输出层的概率最大。Word2vec 词库很大时，训练起来就会很麻烦，训练所需时间就会很长，所以有两种优化方式，hierarchical softmax，利用层次哈夫曼树进行优化，另一种是负采样优化，另外 Skip-gram 模型相对 CBOW 模型在面对生僻词时的表现更好。

Fasttext 模型类似于 Word2vec 的连续词袋模型，不同的是 fasttext 预测句子的标签，而 CBOW 根据周围的词预测中间的词，本文进行漏洞检测，单个语句没有标签。Glove 是通过对词对的共现矩阵进行分解得到的词向量，word2vec 比 glove 更简单，一些研究表明 word2vec 的效果也比 glove 更好 [45]。另外很多漏洞检测方法 [3][8][10][12][13] 均采用 word2vec 对得到的代码特征进行表示学习。所以本文采用 word2vec 作为程序切片表示学习的方法。首先将上文中获得的程序切片进行表示学习，然后用机器学习方法验证其有效性。

程序切片需要向量化才能输入到深度学习模型当中，首先利用词嵌入方法，将源代码中的每一个 token 都转换成词向量，然后将代码中的所有 token 的词向量按顺序拼接，得到整个函数的向量，将函数向量作为机器学习分类器的输入，以验证这种词嵌入方法在本文研究中的有效性。首先介绍 word2vec 的相关概念，然后说明 word2vec 的训练参数，然后介绍用词向量构成输入到 LSTM 和 CNN 中的向量，最后给出机器学习的实验结果。

2.8.1 word2vec 的相关概念

word2vec 是一种将自然语言转换成深度学习模型可接收的作为输入的代表学习方法。该模型以句子作为训练单位，通过对一个双层网络进行训练来得到语料库中每个词的向量，可以学习到词之间的在句子中的上下文语义关系。Word2vec 有两种模型，第一种是 skip-gram 模型，第二种是 CBOW 模型。

比如语料库中的一个句子是 ‘I love China...’，skip-gram 模型根据中心词预测周围的词，假如这里的中心词是 ‘love’，窗口大小为 1，那么双层网络的输入就是 ‘love’ 对应的 one-hot embedding，在输出层，通过 softmax 输出 ‘I’ 和 ‘China’ 等词的概率，通过反复训练使得 ‘I’ 和 ‘China’ 对应的概率最大就完成了训练，此时，网络输入层乘以第一层网络权重就得到了所有词的词向量，也可以说网络的第一层的权重就是词向量。CBOW 模型则是反过来的，它以 ‘I’ 和 ‘China’ 对应的 one-hot embedding 作为网络第一层的输入，输出层输出中心词 ‘love’ 的概率，反复训练使得输出层对应 ‘love’ 的概率最大，和 skip-gram 模型相同，网络第一层的权重为所有词的词向量。

如果语料库太大，就会包括很多词，导致输出层计算概率时复杂度较高，所以有两种优化方法，一种是利用哈夫曼树进行层次 softmax，复杂度变成根号级别。另一种是利用负采样的方式来进行优化。可以发现 skip-gram 模型反向更新时，是多个词去调整网络，而 CBOW 模型是根据一个词的概率去调整网络，面对生僻词时，Skip-gram 模型的效果要优于 CBOW 模型。所以本文

采用的是 word2vec 的 skip-gram 模型。

2.8.2 word2vec 的训练

将上文中得到的切片利用 word2vec 进行训练，学习词向量。首先将切片中的每一条语句进行 token 化，分割成词。然后用 python 的库 gensim 中实现的 Word2vec 训练词向量。Word2vec 中有几个重要参数，min_count 代表词的最小出现频率，如果小于 1，则忽略该词，由于对漏洞检测来说，切片代码中的每一个 token 都很重要，所以在这里取 min_count 等于 1。Sg 参数取 1，代表选择的是 skip-gram 模型。参考 Russell^[7]，Size 参数取 13，代表词向量的维度为 13。

2.8.3 代码向量表示

对于卷积神经网络，输入的是一个二维矩阵，所以对于本文的代码漏洞检测，需要将代码的 token 向量组合成二维矩阵的形式，才能输入到卷积神经网络当中。首先利用 word2vec 得到每个 token 的向量，然后对于切片中每个语句，将其中的每一个 token 向量按顺序拼接组成语句向量，作为二维矩阵的一行，这样一个函数对应的向量就是一个二维矩阵。另外卷积网络的输入是固定的尺寸，所以需要固定每行的向量维度和语句的数量。对于每一种漏洞分别计算数据集中所有函数的语句数的总数，然后除以总的函数数量，计算平均语句数，然后计算所有语句的平均 token 数。以平均语句数作为二维矩阵的长，平均 token 数乘以 token 词向量的维度作为二维矩阵的宽。对于 LSTM 将上面得到的矩阵中的向量，每行按顺序拼接，作为 LSTM 的输入。

2.9 本章小结

本章首先介绍了程序切片的相关概念，包括控制依赖和数据依赖，以及程序依赖图的概念。在此基础上，给出了一段具体的代码例子，详细分析得到程序切片的具体过程，包括找到切片关键点，根据切片关键点得到切片标准，然后用切片标准依据 PDG 中的依赖关系生成程序切片，然后给出从所用代码库中得到的切片的统计信息。最后介绍 word2vec 的概念以及代码向量表示的方法。

第 3 章 算法的实验结果验证

3.1 引言

在上一章中,本文利用 word2vec 对四种漏洞相关的程序切片进行了表示学习,本章先用机器学习方法 SVM,RF 和 LGBM 进行实验,在 ROC_AUC 和 accuracy 两个评估指标上进行评估。另外深度学习对于大量的数据会有更好的表现效果。在漏洞检测领域,已经应用了很多深度学习方法,比如卷积神经网络(CNN),递归神经网络(RNN),双向长短期记忆网络(BiLSTM),都取得了比较好的检测效果,另外也有很多研究引入了注意力机制使漏洞检测的很多指标有所提升。

注意力机制近些年来在深度学习中得到了广泛的应用,在自然语言处理^[35],计算机视觉^[36],模式识别^[37]等各个领域都有很好的表现效果,从注意力机制的命名来看,很容易想到人类的注意力机制,注意力机制借鉴于人眼的观察机制,在人类观察物品时,会特别注意到一些部分,以帮助人类快速获取重要的信息。在深度学习中,注意力机制网络^[38]可以自动关注比较感兴趣的部分,特别关注当前的输入的特征中比较重要的部分,尤其是和当前任务密切相关的,以提高预测的表现效果。比较有代表性的注意力机制源于 Encoder-Decoder 模型,这个模型可以用来进行机器翻译,可以将一句话通过模型转换成另一句话。在翻译多个词时,如果各个词之间的权重保持一致的话,对翻译任务的效果就会有比较大的影响。如果翻译时,更关注对应的词,翻译的结果就会更准确,也就是对应的词的权重要比非对应词的权重更大。通过查询向量 Q , 关键向量 K , 计算注意力分数,对特征向量 V 进行加权,使翻译任务更准确。比如将‘我爱中国’这个句子翻译成英文‘I love China’,将中国翻译成‘China’时,利用到了中文中的全部词语,但应该更关注的是‘中国’,通过注意力机制计算权重分数赋予‘中国’更高的权重。近些年来,自注意力机制(self-attention)也得到了广泛应用,它和之前的注意力机制不同,之前的注意力机制是通过查询向量 Q 计算权重,而自注意力机制可以捕获同一个句子的两个词之间语义信息。虽然 RNN 和 LSTM 也可以捕获句子中的上下文语义信息,但是很难解决长期依赖问题,距离较远的两个词之间的语义信息难以捕获。此时,可以采用自注意力机制。对代码进行漏洞检测时,某种漏洞类型可能更关注切片中的某一条语句,也就是说,不同的语句对漏洞的影响是不同的。在预测漏洞时引入注意力机制,赋予每个词向

量不同的权重，通过实验，观察效果相比没有引入注意力机制时，在各个评价指标上是否有所提升。本文所使用的注意力机制如图 4-5 所示。首先初始化一个全 1 的向量 Q ，连接一个全连接神经网络 N ，然后连接 softmax 层得到一个权重向量 W ，然后将 W 和特征向量 F 相乘，这个特征向量 F 指的是前文中的漏洞检测模型中 softmax 的输入，将权重向量和特征向量相乘，得到加权的特征向量，然后再连接 softmax 进行二分类，关键在于训练过程中，全连接的神经网络 N 的权重在通过反向传播不断优化。所以对于每一种漏洞，对应一种权重。主要是考虑到某一种类型的漏洞可能就会先出现一种语句，再出现另一种语句。

在本章，首先使用机器学习方法进行漏洞检测，然后介绍所使用到的深度学习网络的原理以及进行漏洞检测任务时的模型结构，并将注意力机制引入到漏洞检测模型，进行实验，然后分析实验结果。

随着硬件技术的不断发展，神经网络又重新在人工智能领域复兴起来，深度学习网络训练时通过反向传播，不断优化，调整每层网络之间连接的权重，以在相应的评估指标上有更好的表现。深度学习网络中的双向长短期记忆网络 (BiLSTM) 和卷积神经网络 (CNN) 已经多次应用于代码漏洞检测的领域。所以本文也采用这两种网络进行代码漏洞检测实验。同时，针对应用于文本分类的 CNN 的改进网络 DenseCNN^[17]也进行实验，比对效果。最后加入注意力机制，对比有无注意力机制的检测效果。

3.2 机器学习实验结果

SVM^[30]是利用找到各类样本之间的超平面，对各类数据进行分割的一种方法。在找到最大分割超平面时，分类效果最好。但是有的数据无论怎样在二维平面都找不到一个超平面将各类数据完全分开，通常通过合适的核函数将数据映射到高维空间即可。

随机森林^[31]属于一种集成学习方法^[32]，基学习器是决策树，决策树有三种类型，在随机森林中采取的是决策树的 ID3 树，在节点分裂时，通过删除一部分特征，在剩下的特征中选取分类效果最好的对节点进行分割，引入随机性，能有效降低过拟合。另外随机森林采用的是一种 bagging 的思想，最后的预测结果由多颗决策树投票决定，也能降低过拟合。

随机森林属于集成学习中的 bagging，因为它的最终结果由投票决定。LGBM^[33]也属于集成学习，然而它属于集成学习中的 boosting，是对 GBDT^[34]的一种改进。GBDT 使用的是决策树中的 CART 树，每一轮通过负梯度拟合

损失值，不断优化，它的最终结果是每一颗二叉树 CART 预测结果的叠加，因为每一颗树所拟合的值都是前面所有树的结果的总和和真实值之间的残差，另外为了降低过拟合，每一棵树都会乘上一个权重再进行相加。

上文中对于四种漏洞相关的切片，都利用 word2vec 进行了表示学习，训练得到了词向量，下面利用机器学习方法随机森林(RF)和支持向量机(SVM)以及 LGBM 进行漏洞检测实验，验证学习到的词向量的有效性。

3.2.1 实验数据集

实验数据集选取上一章中得到的切片数据，利用机器学习来验证其有效性。数据不平衡对机器学习有比较大的影响，会导致机器学习分类器的分界面偏向样本数据量较多的一类，导致分类效果不好，在本文中，数据很不平衡，有漏洞的数据远远少于无漏洞的数据，所以给样本少的一类赋予较大的权重，样本数量多的一类赋予较小的权重。实验数据集统计信息如表 2-1 所示。

3.2.2 实验设置

利用 python 的机器学习库 sklearn 实现的 SVM 进行分类，为了解决实验数据不平衡的问题，将 SVM 中的 ‘class_weight’ 参数设置成 balanced，其他参数默认。训练集和测试集比为 7:3。随机森林利用 sklearn 中的 randomforestClassifier 进行实现，‘class_weight’ 参数设置成 balanced，其他参数默认。训练集和测试集比为 7:3。LGBM 利用 python 的机器学习库 lightgbm 实现，训练集和测试集比同为 7:3。

3.2.3 评估指标

采取的评估指标为 ROC_AUC 和准确率，ROC_AUC 是指 roc 曲线（接受者操作特征曲线）下方的面积，这个曲线是根据真阳率（TPR）和假阳率（FPR）画出来的，预测出来的概率在 0 到 1 之间，通过选择不同的阈值计算出不同的 TPR 和 FPR，以画出一条曲线，它的意义就是在预测一个实际为正例的样本时，分类器把它预测为正例的概率。在数据不平衡时，也可以很好的体现出分类器的泛化性能。Accuracy 是指预测的准确率，所有预测正确的样本数量和测试集中样本总数量的比值，可以更直观的体现分类器的泛化性能。

3.2.4 实验结果分析

SVM 针对四种漏洞数据集的实验结果如表 3-1 所示。随机森林针对四种漏洞数据集的实验结果如表 3-2 所示。LGBM 针对四种漏洞数据集的实验结果如表 3-3 所示。在 ROC_AUC 以及准确率两个评估指标上效果都比较好，可以说明 word2vec 有效的学习并表示了切片中的语义信息。可以用来进行漏洞检测。在后面利用深度学习进行漏洞检测时，继续使用 word2vec 作为切片的表示学习方法。

表 3-1 SVM 实验结果

漏洞数据类型	ROC_AUC	accuracy
数组使用相关	0.923	0.948
算术表达式使用相关	0.849	0.849
指针使用相关	0.916	0.879
函数调用相关	0.923	0.921

表 3-2 随机森林实验结果

漏洞数据类型	ROC_AUC	accuracy
数组使用相关	0.924	0.954
算术表达式使用相关	0.743	0.919
指针使用相关	0.846	0.912
函数调用相关	0.910	0.899

表 3-3 LGBM 实验结果

漏洞数据类型	ROC_AUC	accuracy
数组使用相关	0.930	0.953
算术表达式使用相关	0.743	0.919
指针使用相关	0.846	0.912
函数调用相关	0.905	0.934

在上一章利用 CPP 代码分析工具 Joern 得到了四种漏洞相关的切片的基础上，首先对机器学习方法支持向量机，随机森林以及 LGBM 的原理进行简要介绍，最后使用 word2vec 得到的向量利用机器学习方法随机森林（RF），支持向量机（SVM）以及 LGBM 进行漏洞检测，通过实验结果验证学习到的向量的有效性，将其应用于后续的深度学习模型进行漏洞检测。

3.3 基于 BiLSTM 的代码漏洞检测模型

传统的全连接神经网络不能处理序列数据，不能捕捉到序列数据中的语义信息。所以出现了递归神经网络 RNN，RNN 中有时序的概念，当前时刻的值依赖于上一时刻的隐藏层状态，并且当前时刻的隐藏层状态会影响到下一时刻的值，这样的结构和特点就很适合处理序列数据。由于反向传播更新权重，在序列长度过长时，会产生梯度消失或者梯度爆炸的问题，甚至导致网络权重无法更新，影响网络学习的效果。在后来出现的长短期记忆网络（LSTM）中，利用细胞元以及门结构，在门结构内部通过输入门，遗忘门，输出门三个门结构缓和了递归神经网络的长期依赖问题。

其中遗忘门的输入包括两部分，一部分是时间步 t 的隐藏层状态 h_{t-1} 和当前时间步的输入 x_t ，利用 sigmoid 函数输出当前时间步遗忘前面传来信息的概率，遗忘门的计算方法如公式 3-1 所示。

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (3-1)$$

输入门也包含两个部分的输入，和遗忘门的输入相同，输入门也是用 sigmoid 函数，用来决定当前输入信息有多少的比例加入到细胞状态中，输入门的相关表达式如公式 3-2 和公式 3-3 所示。

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3-2)$$

$$C\tilde{t} = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (3-3)$$

遗忘门输出的概率 f_t 和前一时间步的细胞状态 C_{t-1} 相乘，对细胞状态进行一部分更新，另外输入门输出的概率 i_t 和输入门得到的另一个输出 $C\tilde{t}$ 相乘的结果作为细胞状态的另一部分更新，细胞状态相关的数学公式如公式 3-4 所示。

$$C_t = f_t * C_{t-1} + i_t * C \quad (3-4)$$

输出门需要输出当前时间步的输出 o_t ，另一部分是输出当前隐藏层的状态 h_t ，相关公式见公式 3-5 和公式 3-6。

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o) \quad (3-5)$$

$$h_t = o_t * \tanh(C_t) \quad (3-6)$$

双向 LSTM 就是两个方向的 LSTM 的叠加，可以获取两个方向的语义信息。利用双向 LSTM 来进行漏洞检测的总体结构如图 3-1 所示，每个时间步对应一个 token 的词向量，针对每一种类型的切片单独训练一个网络，BiLSTM 的输出连接全连接层，然后连接 softmax 层进行二分类，判断当前切片是否存在该类型的漏洞，属于一个二分类的任务。

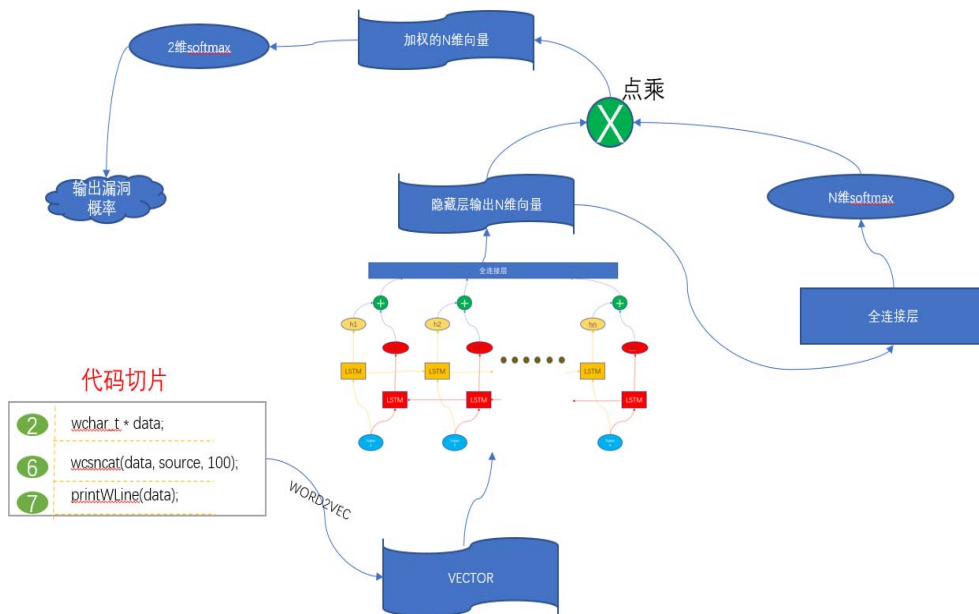


图 3-1 BiLSTM 漏洞检测结构图

3.4 基于 TextCNN 的代码漏洞检测模型

CNN 常常应用于图像处理领域，因为图像是矩阵的形式，通过卷积和池化操作获取图像的不同部位，对图像进行分类。Kim^[39]等人在做文本分类时，提出了 TextCNN，即用 CNN 对文本进行分类。TextCNN 的结构如图 3-2 所示。

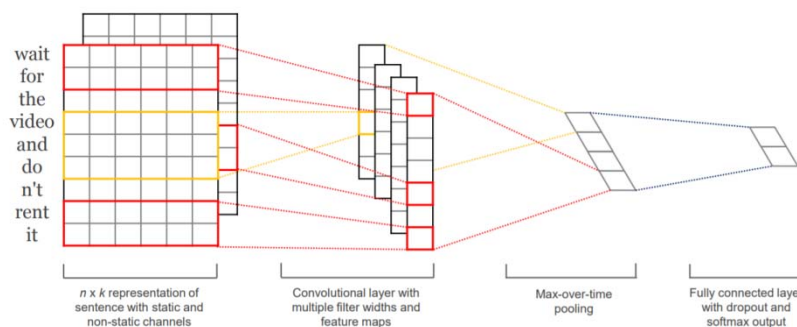


图 3-2 TextCNN 结构图

可以看出，TextCNN 和传统 CNN 类似，都要以矩阵作为网络第一层的输入，传统 CNN 是对图片的二维矩阵进行处理，为了能够处理文本，需要将文本转换为矩阵，从图中可以看出，是将每一个词作为一行，那么一句话就可以是多行，从而组成一个矩阵。这里的每一个词利用 word2vec 训练的词向

量就作为矩阵中的一行，然后就可以输入到 CNN 中，对其进行卷积和池化操作。但是和传统 CNN 不同的是，文本数据并不适用于横向卷积，在图像数据中，一行不是完整的一个部分，是可以横向卷积的，也可以获取到图片中的信息，但是文本数据对应的矩阵，一行对应一个词的向量，横向卷积会割裂一个词的完整性，导致横向卷积获取到的信息不完整，所以卷积核的宽度和词向量的维度应该保持一致，卷积核的长度可以变化，不同的长度就代表考虑不同个词数的上下文语义信息，所以可以进行纵向卷积。例如在图 3-2 中，取卷积核长度为 2，则卷积时考虑的上下文语义信息就是每两个词之间的。

同时在 Text CNN 中可以考虑同时使用多个不同的卷积核，即多个长度不同，宽度均为词向量维度的卷积核，这样就可以多考虑不同个连续的词之间包含的语义信息，类似于 n-gram 取不同的 n，使预测效果更加准确。为了便于处理，多个不同的卷积核对同一矩阵卷积之后得到的特征图的形状是不同的，所以在卷积后，对每一个特征图进行最大池化，提取其中最重要的信息，得到一维向量，然后将所有最大池化的向量结果进行拼接，拼接后连接全连接层，最后接 softmax 层，进行分类。

TextCNN 可以应用于文本分类，主要是因为在卷积时，没有割裂一个词的完整性，并且由于存在多个长度不同的卷积核，网络学习到了文本的上下文语义信息。

将 TextCNN 应用于漏洞检测时，在上文的基础上需要有一些改变，如果以代码语句的每一个 token 对应的向量作为一行，那么卷积时，获得的语义信息就是一个语句中的某几个 token，不能完整的表达语句的语义信息，将完整的语句割裂了。而且一个函数中包含的 token 过多，如果全都保留，卷积神经网络的输入就过大，对实验设备的内存有效高的要求。如果不全都保留，损失信息过多。所以本文将每行语句包含的 token 对应的向量按顺序拼接，由词向量组成句子向量，然后一个句子的向量代表矩阵的一行，矩阵的长度代表函数的语句数。由于卷积神经网络的输入尺寸是固定的，所以需要固定代码的输入大小，同时为了避免损失信息过多，使预测的效果较好。采取数据集中所有函数的平均语句数作为 TextCNN 网络的输入矩阵的长度，采取数据集中所有函数包括的所有语句数的平均 token 数乘以词向量维度的值作为 TextCNN 网络的输入矩阵的宽度。TextCNN 漏洞检测结构如图 3-3 所示。卷积的过程相当于降维，比如卷积层的输入形状为 $h*w*d$ ，filter 的数量为 K，filter 的大小为 F，步长为 S，padding 大小为 P，则输出体积为 $H*W*D$ ，相关计算公式如公式 3-7，3-8，3-9 所示。

$$H = \frac{h-F+2P}{S} + 1 \quad (3-7)$$

$$W = \frac{w-F+2P}{S} + 1 \quad (3-8)$$

$$D = K \quad (3-9)$$

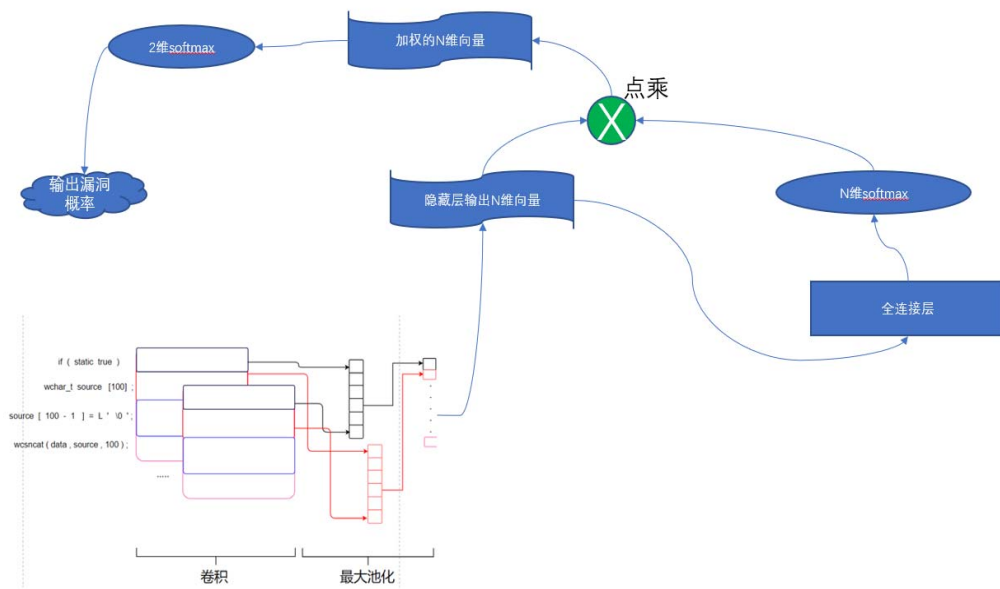


图 3-3 TextCNN 漏洞检测结构图

3.5 基于 DenseCNN 的代码漏洞检测模型

传统的神经网络有 N 层，那么就有 N 个两层之间的相关连接，每一层直接和它的后一层相连。随着硬件技术的发展，CNN 网络的层数不断加深，但是特征也有可能在深层次的网络传递中被冲刷掉。在图像识别领域中，出现了 ResNets^[40]，以及 Highway^[41]网络经过身份连接和旁路信号来解决 CNN 梯度消失问题。Stochastic depth(随机深度)^[42]通过随机丢掉网络中的一些中间层来缓和梯度消失，同时实现更好的特征传递。这些方法都是使卷积神经网络的输入层和输出层之间的距离变短，DenseCNN 的思想类似，DenseCNN^[17]是指 Densely Connected Convolutional Networks，为稠密连接的卷积神经网络，是对普通的卷积神经网络的一种改进，它以向前推进的方式连接各层。DenseCNN 有 $[N * (N + 1)] / 2$ 个连接。对于每一层，它之前的所有层输出的特征图 (feature-map) 都作为本层的输入，也就是说第 L 层有 L 个输入。而本层的输出特征图作为它后面所有层的输入。这样的连接方式可以缓和梯度消失问题，加强了特征的传递，有利于特征的重复利用，并减少了参数^[17]。DenseCNN 漏洞检测结构图如图 3-4 所示。假设 x_0 是第一层卷积层的输入，

$H_\ell(\cdot)$ 是一个复合函数，那么第 L 层的输入如公式 4-10 所示。

$$x_\ell = H_\ell([x_0, x_1, \dots, x_{\ell-1}]) \quad (3-10)$$

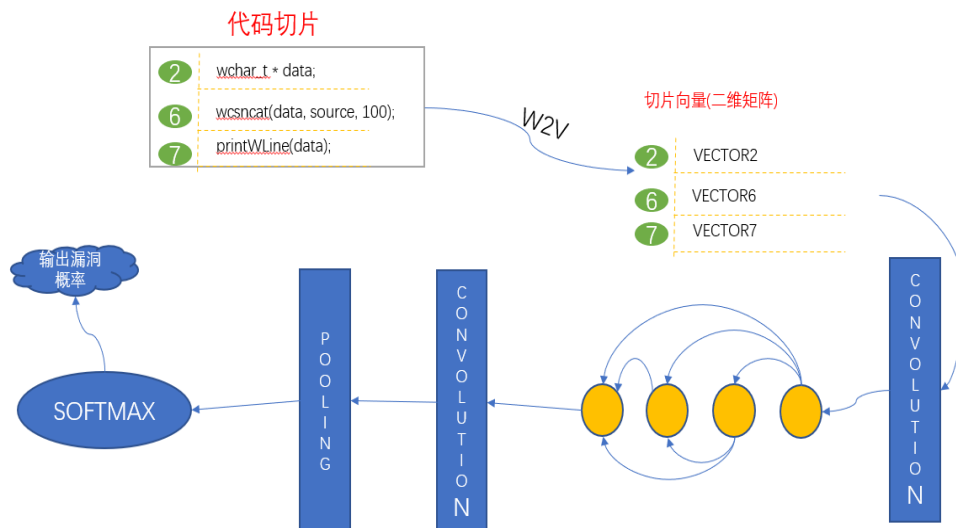


图 3-4 DenseCNN 漏洞检测结构图

3.6 实验结果与分析

在前文中简要介绍了深度学习网络以及注意力机制，并且在得到了切片以及通过切片表示学习得到向量的基础上，本节利用深度学习网络和注意力机制进行漏洞检测实验，首先对实验数据进行简要介绍，然后对评价指标进行简要说明，最后对得到的实验结果进行分析。

3.6.1 数据集信息

数据集采用的是 MVD 数据集^[6]的源代码，该源代码源于漏洞数据库 NVD 和 SARD^[44]，本文使用的是其中带有 CWE 标签的源文件，因为这部分源文件便于打标签，对于里面的函数，都是以 bad 和 good 命名的，bad 对应的函数为有漏洞的函数，good 对应的函数为无漏洞的函数，并且对于 bad 函数，源代码中通过注释标明了其漏洞语句代码所在位置，即在源文件中的行号。利用 CPP 分析工具 Joern 得到切片之后，首先要对切片打标签，此时需要遍历切片中的每一条语句，判断是否在源代码中是漏洞语句，所以，找到切片语句在源代码中的代码行，判断该代码行上方是否有注释‘FLAW’，如果有，就停止遍历切片中的其余代码并将标签标记为 1，否则，继续遍历切片的代码中的剩余代码，最后切片标签标记为 0。切片后的统计信息如表 2-1 所

示。

3.6.2 实验设置

本节介绍所有的实验设备，参数值，以及环境，安装包版本等配置信息。实验设备采用的是联想笔记本电脑，具体信息如表 3-4 所示。

表 3-4 实验设备信息

配置名称	配置型号
笔记本型号	Y7000P
显卡	GTX2060
内存	16G
CPU	i7-9750H
操作系统	Windows 10 home
Python 版本	3.7

对程序进行切片需要用到 Joern 工具，Joern 需要用到 linux 环境，利用虚拟机实现，虚拟机和相关配置信息如表 3-5 所示。

表 3-5 虚拟机相关信息

配置名称	配置型号
虚拟机版本	Vmware workstation 15.5 Pro
Linux 版本	Ubuntu18.04.4
内存	8G
Python 版本	2.7

切片需要用到 CPP 分析工具 Joern, Joern 的相关配置信息如表 3-6 所示。Neo4j 是一种 NOSQL 型的高性能图像数据库，可以用来存储 Joern 分析得到的控制依赖图（CFG），程序依赖图（PDG）等图形信息，在图中存在节点和关系，而节点和关系都分别存在键和对应的值。Gremlin 是一种图数据库的查询语言，在本文当中用来在程序依赖图上查询所需要的节点，获取对应节点的属性，包括代码行，源代码等信息。Python-joern 是 joern 的 python 接口，方便利用 python 语言实现对 Joern 的操作。Joern-tools 是 joern 所使用到的一些相关的工具包。另外 Joern 是用 java1.7 编写的，所以需要用到 JDK1.7 才

能正确使用 Joern 工具对 CPP 代码进行分析。

表 3-6 Joern 相关信息

配置名称	参数值/配置版本
Joern 版本	0.3.1
Neo4j 版本	Community-2.1.5
Gremlin	8G
Python-joern	0.3.1
Joern-tools	0.3.1
JDK	1.7.0_79

3.6.3 评估指标

在用机器学习方法验证用 word2vec 学到的词向量的有效性时，只用到了 ROC_AUC 和准确率两个评估指标，初步的验证了利用 word2vec 学习切片特征向量的有效性，在本章，用更多的评估指标更全面的对漏洞检测结果进行评估。采用了 ROC_AUC，假阳率（FPR），假阴率（FNR），真阳率（TPR），召回率（PR），准确率（ACC）等评价指标。

ROC_AUC 是指接受者操作特征曲线下方的面积，以 FPR 为横轴，以 TPR 为纵轴，绘制接受者操作特征曲线，因为接受者操作特征曲线的绘制是以不同的正样本阈值来画出，所以对数据不平衡时的模型评估效果比较准确，具体意义是指，当一个样本实际为正样本时，当前被评估的模型将该样本预测为正样本的概率。本文利用机器学习库 sklearn 实现 ROC_AUC 的计算。

假阳率是指实际为负例，预测为正例的样本数占有所有实际为负例的样本数的比例。该比值越高表明模型越偏向于将负例预测为正例。该比值越低越好。

假阴率是指实际为正例，预测为负例的样本数占有所有实际为正例的样本数的比例。该比值越高表明模型越偏向于将正例预测为负例。该比值越低越好。

真阳率（recall）是指实际为正例，预测为正例的样本数占有所有实际为正例的样本数的比例。该比值越高表明模型召回率越高，检测漏洞的能力越强，该比值越高越好。

精度是指实际为正例，预测为正例的样本数占有所有预测为正例的样本数

的比例。该比值标明的是检测漏洞的精度，越高越好。

F1-score 是 precision 和 TPR (recall) 的调和平均，越高越好。

G-measure 也是 precision 和 TPR 的一种平均，越高越好。

准确率表明模型预测正确的样本数占总样本数的比例，表明模型预测的准确率，越高越好。

全部的评估指标以及其计算公式如表 3-7 所示。

表 3-7 评估指标

评估指标	计算方法
ROC_AUC	Roc 曲线下方面积
FPR	$fp / (fp + tn)$
FNR	$fn / (fn + tp)$
TPR (recall)	$tp / (tp + fn)$
Precision	$tp / (tp + fp)$
F1-score	$(2 * precision * recall) / (precision + recall)$
G-measure	$pow((precision * recall), 0.5)$
Accuracy	$(tp + tn) / (tp + fp + tn + fn)$

3.6.4 实验结果及分析

在前文的基础上，得到了四种漏洞的切片数据集，针对每一种漏洞数据集单独进行实验。然后用数组使用相关的漏洞数据训练 CNN 模型，用指针使用相关和算术表达式相关这两种漏洞测试模型，观察跨类型的检测效果。最后将四种类型的漏洞数据混合到一起训练，测试。本小节给出实验结果以及实验中一些关键参数的设置，并对实验结果进行分析。

(1) 数组使用相关漏洞的检测实验：该实验数据中，切片共有 12437 个切片，其中无漏洞的切片数量为 10760，有漏洞的切片数量为 1677。在前文中已经将切片转换成向量，但是对于 CNN 需要固定大小的输入。所以计算每个函数的平均代码行数 `ave_line`，并计算每个语句的平均 token 数 `ave_token`，将平均代码行数作为 CNN 输入矩阵的长度，将平均 token 数作为 CNN 输入矩阵的宽度。针对数组使用相关的漏洞 `ave_line` 为 7.78，取为 8。`ave_token`

为 6.315 取为 7。然后利用训练集和验证集 7:3 对模型进行训练，并利用 early_stopping 降低过拟合。具体参数图表 3-8 所示。其中 early_stopping 以验证集的 loss 为指标，当连续十个 epoch 发现验证集的 loss 不再下降时，就提前停止训练。然后用测试集对模型进行测试，给出测试结果，其中训练集和验证集的总数和测试集比为 7:3。

表 3-8 数组使用相关漏洞实验参数

参数名称	参数值
优化器 Adam	Lr=0.001
Loss 损失函数	Binary_crossentropy
metrics	accuracy
Early_stopping metrics	Val_loss
Early_stopping epoch	20
Epoch	50

数组使用相关漏洞实验结果 如表 3-9 所示。

表 3-9 数组使用相关漏洞实验结果

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9810	0.9826	0.9803	0.9802	0.9821
FPR	0.0012	0.0031	0.0059	0.0087	0.0009
FNR	0.1767	0.1631	0.1650	0.1476	0.1748
TPR(recall)	0.8233	0.8369	0.8349	0.8524	0.8252
precision	0.9907	0.9773	0.9577	0.9500	0.9930
F1-score	0.8992	0.9017	0.8921	0.8941	0.9014
G-measure	0.9031	0.9044	0.8942	0.8952	0.9052
Accuracy	0.9745	0.9748	0.9721	0.9721	0.9751

观察数组使用相关的漏洞的实验结果，可以发现针对三种模型，有无注意力机制，提升并不明显，甚至有的指标出现了下降，说明本来的模型已经从特征中学到了足够多的信息，另外假阴率指标比较高，说明漏报率比较高。精度，ROC_AUC 以及准确率三个指标比较好，表明准确率比较高，并且精

度比较高，说明误报率很低。漏报率高应该是取平均语句数和平均 token 数时造成了很多的信息损失，刚好损失掉了漏洞语句的部分，所以导致漏报率较高。

(2) 算术表达式使用相关漏洞的检测实验：该实验数据中，切片共有 60292 个切片，其中无漏洞的切片数量为 53082，有漏洞的切片数量为 7210。在前文中已经将切片转换成向量，但是对于 CNN 需要固定大小的输入。所以计算每个函数的平均代码行数 `ave_line`，并计算每个语句的平均 token 数 `ave_token`，将平均代码行数作为 CNN 输入矩阵的长度，将平均 token 数作为 CNN 输入矩阵的宽度。针对数组使用相关的漏洞 `ave_line` 为 11.29，取为 12。`ave_token` 为 6.97 取为 7。然后利用训练集和验证集 7:3 对模型进行训练，并利用 `early_stopping` 降低过拟合。具体参数如表 3-10 所示。其中 `early_stopping` 以验证集的 loss 为指标，当连续十个 epoch 发现验证集的 loss 不再下降时，就提前停止训练。然后用测试集对模型进行测试，给出测试结果，其中训练集和验证集的总数和测试集比为 7:3。

表 3-10 算术表达式使用相关漏洞实验参数

参数名称	参数值
优化器 Adam	Lr=0.001
Loss 损失函数	Binary_crossentropy
metrics	accuracy
Early_stopping metrics	Val_loss
Early_stopping epoch	20
Epoch	50

算术表达式使用相关漏洞实验结果 如表 3-11 所示。

表 3-11 算术表达式使用相关漏洞实验结果

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9767	0.9784	0.9820	0.9811	0.9811
FPR	0.0083	0.0097	0.0119	0.0127	0.0227
FNR	0.2190	0.2140	0.1947	0.1942	0.1460
TPR(recall)	0.7809	0.7859	0.8053	0.8058	0.8540
precision	0.9292	0.9189	0.9039	0.8985	0.8404
F1-score	0.8487	0.8472	0.8518	0.8496	0.8471
G-measure	0.8519	0.8498	0.8532	0.8509	0.8471
Accuracy	0.9658	0.9652	0.9656	0.9650	0.9622

针对算术表达式相关漏洞的实验结果，三种网络模型，以及有无注意力机制差别不大，提升不明显，说明原本的网络已经从特征中学到了足够的信息。另外漏报率较高，都在 19%左右，最高为 21.90%，最低为 18.53%。因为截断函数和函数语句有一定的信息损失。

(3) 指针使用相关漏洞的检测实验：该实验数据中，切片共有 13455 个切片，其中无漏洞的切片数量为 10799，有漏洞的切片数量为 2676。在前文中已经将切片转换成向量，但是对于 CNN 需要固定大小的输入。所以计算每个函数的平均代码行数 `ave_line`，并计算每个语句的平均 token 数 `ave_token`，将平均代码行数作为 CNN 输入矩阵的长度，将平均 token 数作为 CNN 输入矩阵的宽度。针对数组使用相关的漏洞 `ave_line` 为 5.58，取为 6。`ave_token` 为 6.18 取为 7。然后利用训练集和验证集 7:3 对模型进行训练，并利用 `early_stopping` 降低过拟合。具体参数图表 3-12 所示。其中 `early_stopping` 以验证集的 loss 为指标，当连续十个 `epoch` 发现验证集的 loss 不再下降时，就提前停止训练。然后用测试集对模型进行测试，给出测试结果，其中训练集和验证集的总数和测试集比为 7:3。

表 3-12 指针使用相关漏洞实验参数

参数名称	参数值
优化器 Adam	Lr=0.001
Loss 损失函数	Binary_crossentropy
metrics	accuracy
Early_stoping metrics	Val_loss
Early_stoping epoch	20
Epoch	50

指针使用相关漏洞实验结果 如表 3-13 所示。

表 3-13 指针使用相关漏洞实验结果

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9917	0.9901	0.9949	0.9948	0.9903
FPR	0.0253	0.0077	0.0213	0.0127	0.0139
FNR	0.0848	0.1496	0.0723	0.0985	0.1060
TPR(recall)	0.9152	0.8504	0.9277	0.9014	0.8940
precision	0.8995	0.9646	0.9151	0.9463	0.9409
F1-score	0.9073	0.9039	0.9214	0.9234	0.9169
G-measure	0.9073	0.9057	0.9214	0.9236	0.9172
Accuracy	0.9628	0.9641	0.9685	0.9703	0.9678

针对指针使用相关漏洞的实验结果，可以发现，除和前两个实验一样，在 ROC_AUC，精度和准确率上表现较好以外，漏报率也有所降低，最高 14.96%，最低 7.23%。

(4) 函数调用相关漏洞的检测实验：该实验数据中，切出共有 16720 个切片，其中无漏洞的切片数量为 13809，有漏洞的切片数量为 2911。在前文中已经将切片转换成向量，但是对于 CNN 需要固定大小的输入。所以计算每个函数的平均代码行数 `ave_line`，并计算每个语句的平均 token 数 `ave_token`，将平均代码行数作为 CNN 输入矩阵的长度，将平均 token 数作为 CNN 输入矩阵的宽度。针对数组使用相关的漏洞 `ave_line` 为 4.10，取为 5。`ave_token`

为 5.95 取为 6。然后利用训练集和验证集 7:3 对模型进行训练，并利用 early_stopping 降低过拟合。具体参数图表 4-23 所示。其中 early_stopping 以验证集的 loss 为指标，当连续十个 epoch 发现验证集的 loss 不再下降时，就提前停止训练。然后用测试集对模型进行测试，给出测试结果，其中训练集和验证集的总数和测试集比为 7:3。

表 3-14 函数调用相关漏洞实验参数

参数名称	参数值
优化器 Adam	Lr=0.001
Loss 损失函数	Binary_crossentropy
metrics	accuracy
Early_stopping metrics	Val_loss
Early_stopping epoch	20
Epoch	50

函数调用相关漏洞实验结果 如表 3-15 所示。

表 3-15 函数调用相关漏洞实验结果

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9908	0.9891	0.9930	0.9937	0.9936
FPR	0.0091	0.0208	0.0117	0.0096	0.0069
FNR	0.1246	0.0925	0.1139	0.1127	0.1329
TPR(recall)	0.8754	0.9075	0.8861	0.8873	0.8671
precision	0.9510	0.8979	0.9384	0.9492	0.9618
F1-score	0.9117	0.9027	0.9115	0.9172	0.9120
G-measure	0.9125	0.9027	0.9119	0.9177	0.9133
Accuracy	0.9715	0.9671	0.9711	0.9731	0.9719

观察函数调用相关漏洞实验结果，可以发现同样是漏报率比较高，其它指标相对较好，三种模型以及注意力机制对结果影响不大，说明网络已经学习到了足够的信息。

(4) 混合所有数据进行实验：本节将所有数据混合到一起，对模型进行

训练和测试。共 102904 个程序切片，正例有 14474，负例有 88430。训练集和验证集比为 7:3，训练集和验证集的总数和测试集比为 7:3。实验结果如表 3-16 所示。实验结果表明可以将所有数据混合进行漏洞检测。

表 3-16 混合数据实验结果

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9886	0.9880	0.9886	0.9888	0.9881
FPR	0.0060	0.0059	0.0135	0.0125	0.0060
FNR	0.1436	0.1459	0.1133	0.1317	0.1532
TPR(recall)	0.8564	0.8541	0.8867	0.8683	0.8468
precision	0.9588	0.9593	0.9136	0.9180	0.9577
F1-score	0.9048	0.9036	0.9000	0.8924	0.8988
G-measure	0.9062	0.9051	0.9000	0.8928	0.9005
Accuracy	0.9746	0.9744	0.9727	0.9710	0.9736

(6) 实验结果分析：利用 BiLSTM，TextCNN 以及 DenseCNN 三种网络共进行了包括针对数组使用相关漏洞，算术表达式使用相关漏洞，指针使用相关漏洞，函数调用相关漏洞四种数据以及混合数据集的实验。

可以发现在所有漏洞的实验中，在假阴率这个指标上效果不是很好，说明漏洞的漏报率比较高，同时，除了指针使用相关漏洞和算术表达式相关漏洞的精度，所有实验在精度，准确率和 ROC_AUC 三个指标上表现都很好，都在 95%以上，因此还可以通过提取更多的特征和采用更适合的深度学习网络来进一步降低。另外多个实验之间的评估指标差距不是很大，说明需要进一步提取出对漏洞更具有代表性的特征以提高检测表现。分析漏报率较高，是因为，为了固定卷积神经网络的输入，对语句数取了所有函数代码行的平均数，对于每一行的 token 数，取了所有语句的平均 token 数，准确率和精度较高说明误报率较低，表明网络学到了区分有无漏洞代码之间的区别，

利用四种漏洞数据集，将最大 token 数和最大语句数取之前实验中的平均 token 数和平均语句数。统计数据集中的所有函数，所有函数中，最多包含 54 行语句，所有代码行中，包含最多 40 个 token。词向量为 13 维。利用三种网络分别进行实验，实验结果如表 3-17-3-20 所示，混合数据实验如表 3-21 所示。各个指标都有了明显的提升，尤其是漏报率，并且其它指标上的效果也比较好，证明了程序切片的有效性，另外相比不能处理序列信息的机

器学习方法，三种深度学习网络都取得了更好的表现效果，表明对于代码漏洞检测任务，通过学习其上下文语义信息，能提高检测的准确率。

表 3-17 数组使用相关漏洞检测实验

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9867	0.9877	0.9979	0.9978	0.9904
FPR	0.0074	0.0049	0.0052	0.0071	0.0028
FNR	0.0441	0.0651	0.0336	0.0357	0.0924
TPR(recall)	0.9559	0.9349	0.9664	0.9643	0.9076
precision	0.9499	0.9653	0.9644	0.9523	0.9796
F1-score	0.9529	0.9498	0.9654	0.9582	0.9422
G-measure	0.9529	0.9500	0.9654	0.9583	0.9429
Accuracy	0.9879	0.9874	0.9912	0.9893	0.9858

表 3-18 算术表达式相关漏洞检测实验

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9922	0.9923	0.9966	0.9955	0.9934
FPR	0.0154	0.0046	0.0067	0.0059	0.0021
FNR	0.0677	0.1275	0.0531	0.0624	0.1580
TPR(recall)	0.9323	0.8725	0.9469	0.9376	0.8420
precision	0.8966	0.9648	0.9532	0.9579	0.9830
F1-score	0.9141	0.9163	0.9500	0.9477	0.9070
G-measure	0.9142	0.9175	0.9500	0.9477	0.9097
Accuracy	0.9780	0.9800	0.9875	0.9870	0.9783

表 3-19 指针使用相关漏洞检测实验

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9949	0.9952	0.9974	0.9971	0.9981
FPR	0.0106	0.0062	0.0320	0.0115	0.0109
FNR	0.0548	0.0670	0.0146	0.0365	0.0414
TPR(recall)	0.9452	0.9330	0.9854	0.9635	0.9586
precision	0.9580	0.9746	0.8871	0.9553	0.9574
F1-score	0.9516	0.9533	0.9336	0.9594	0.9580
G-measure	0.9516	0.9536	0.9349	0.9594	0.9580
Accuracy	0.9804	0.9814	0.9715	0.9834	0.9829

表 3-20 函数调用相关漏洞检测实验

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9976	0.9947	0.9984	0.9984	0.9983
FPR	0.0060	0.0055	0.0041	0.0046	0.0055
FNR	0.0116	0.0151	0.0093	0.0093	0.0081
TPR(recall)	0.9884	0.9849	0.9907	0.9907	0.9919
precision	0.9714	0.9736	0.9804	0.9782	0.9737
F1-score	0.9798	0.9792	0.9855	0.9844	0.9827
G-measure	0.9799	0.9792	0.9856	0.9844	0.9828
Accuracy	0.9930	0.9922	0.9950	0.9946	0.9940

表 3-21 混合数据漏洞检测实验

评估指标	TextCNN	TextCNN+atten	BiLSTM	BiLSTM+atten	DenseCNN
ROC_AUC	0.9876	0.9879	0.9920	0.9917	0.9910
FPR	0.0137	0.0156	0.0206	0.0125	0.0137
FNR	0.1041	0.0929	0.0818	0.0855	0.0892
TPR(recall)	0.8959	0.9071	0.9182	0.9145	0.9108
precision	0.9163	0.9070	0.8821	0.9248	0.9176
F1-score	0.9060	0.9071	0.8998	0.9196	0.9142
G-measure	0.9061	0.9071	0.8999	0.9196	0.9142
Accuracy	0.9733	0.9733	0.9706	0.9770	0.9754

如图 3-5 到 3-8 为四种漏洞的误报和漏报折线图，3-9 为混合数据的折线图。可以看出对于每一种漏洞来说，除了用 DenseCNN 预测算术表达式漏洞，其余结果都是取最大 token 数和最大语句数的结果在漏报率上有很大的降低。在函数调用和算术表达式两种漏洞的实验中，误报率也有所降低。

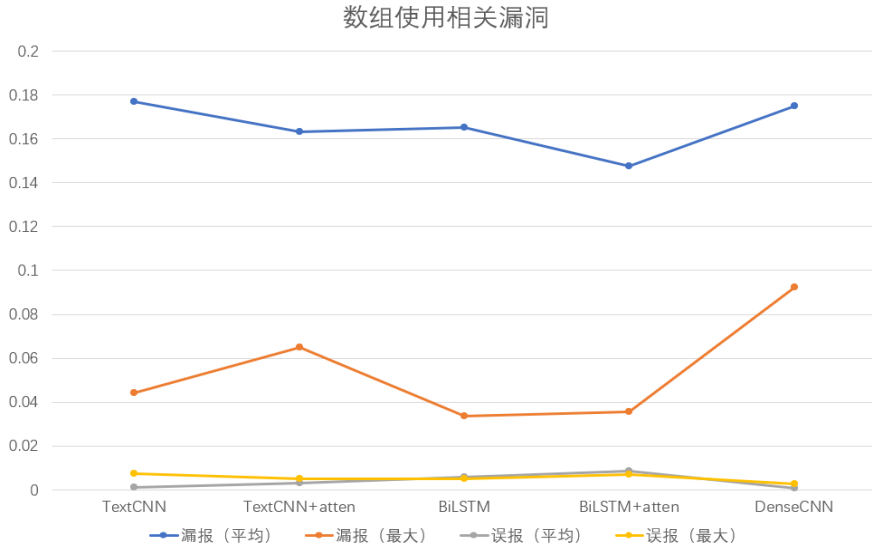


图 3-5 数组使用相关漏洞误报漏报折线图

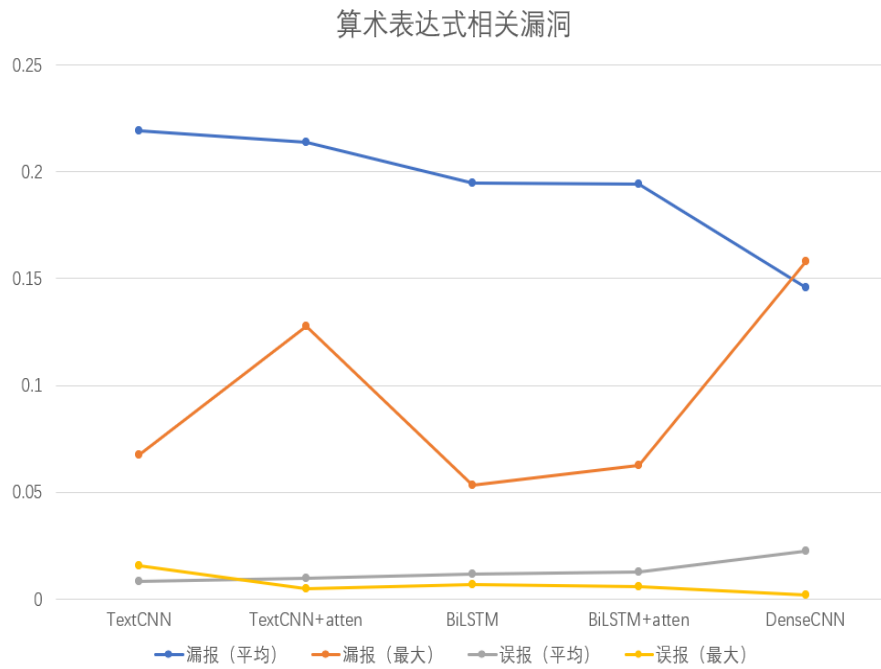


图 3-6 算术表达式相关漏洞误报漏报折线图

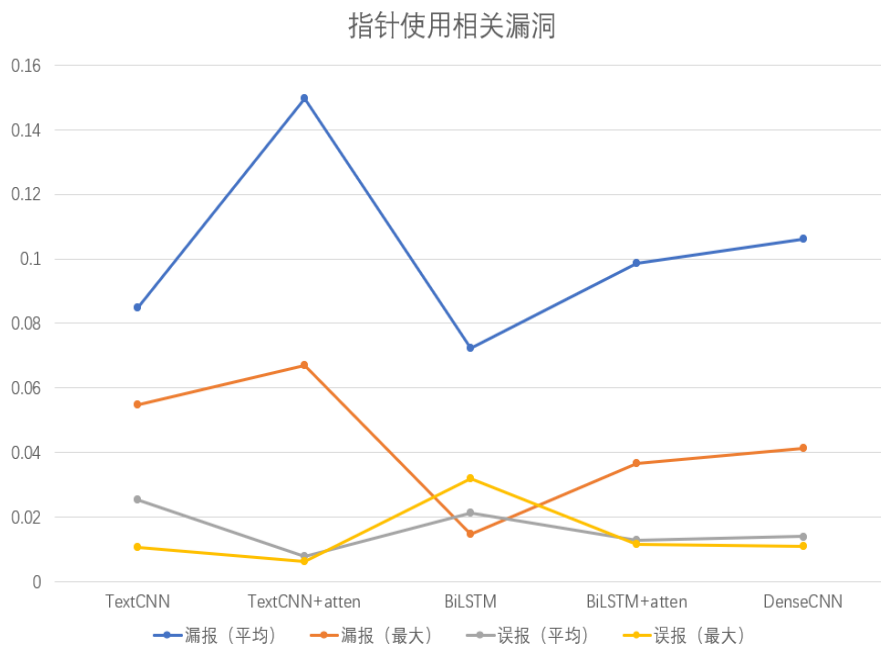


图 3-7 指针使用相关漏洞误报漏报折线图

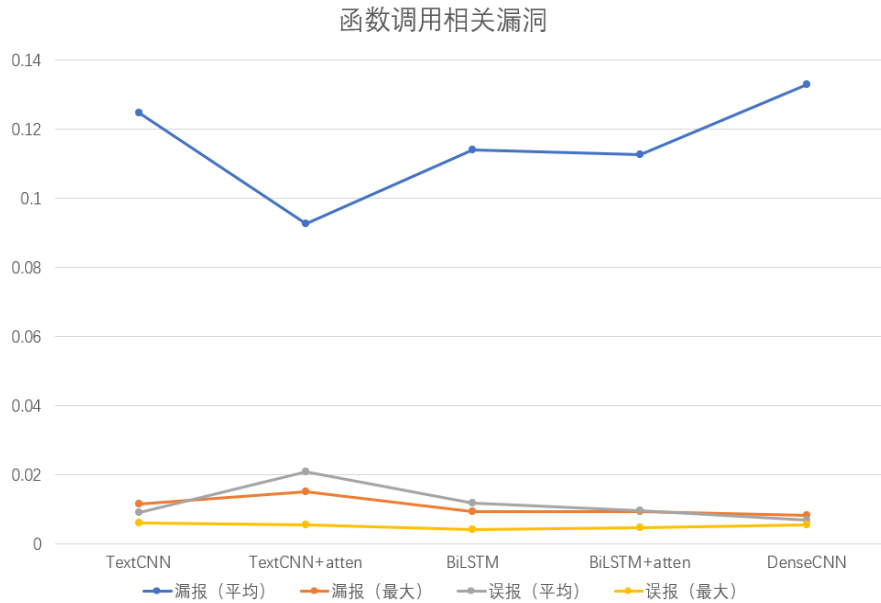


图 3-8 函数调用相关漏洞误报漏报折线图

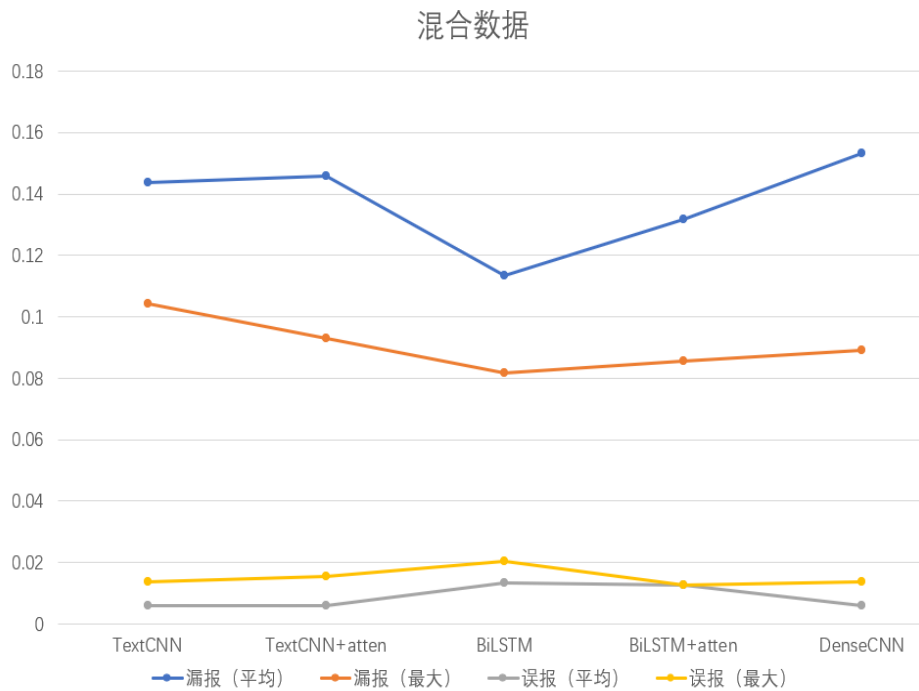


图 3-9 混合数据漏洞检测误报漏报折线图

3.7 本章小结

本章首先对本文采用的代码漏洞检测模型进行介绍，分别介绍了 CNN，TextCNN 以及 BiLSTM 的原理和网络结构图，并进一步给出了本文进行漏洞

检测的网络结构图，然后简单介绍了注意力机制的原理以及应用于本文的注意力机制。

然后介绍了进行实验的数据集的相关信息，以及实验环境，各种配置文件，相关工具和开发语言的版本。然后详细介绍了所使用的评估指标的含义以及计算公式。最后给出针对数组使用相关，算术表达式使用相关，指针使用相关，函数调用相关四种数据集在 TextCNN，BiLSTM 以及 DenseCNN 三种网络下的实验结果。并给出有无注意力的实验结果对比。最后对实验结果进行分析。

结 论

软件已经成为人类生活中必不可少的一部分，但是软件漏洞却让一些恶意攻击人员有了可乘之机，严重危害了软件的使用体验以及社会公共安全。近些年来出现了很多利用深度学习进行漏洞检测的研究，本文研究了多个漏洞检测方法，以及漏洞检测所使用的特征和特征的向量化方法。同时也对代码的中间表示方法，程序切片进行了深入的研究，并利用深度学习方法进行漏洞检测。

针对基于代码语义向量和深度学习的代码漏洞检测，现研究成果如下：

利用四种漏洞关键点源代码进行切片，进行切片级别的代码漏洞检测，检测粒度比较细，更有助于代码修复人员定位漏洞代码。

(1) 根据切片准则，用 `joern` 工具分析源代码，得到了一个具有四种切片的二分类漏洞数据集，并且每个切片都标记为有漏洞或者无漏洞。

(2) 针对四种漏洞关键点，获得程序切片，分别进行实验。本文提出使用 `DenseCNN` 以及 `TextCNN` 进行漏洞检测，并将 `token` 向量拼接成句子向量，作为卷积网络输入矩阵中的一行向量，漏洞检测取得了较好的效果。另外在 `TextCNN` 和 `BiLSTM` 中引入了自注意力机制，使漏洞检测在一部分指标上进一步提升，除了指针使用相关漏洞和算术表达式相关漏洞的精度，所有实验在精度，准确率和 `ROC_AUC` 三个指标上表现都很好，都在 95% 以上。

当前存在的不足以及未来的工作：

(1) 当前进行的实验在 `FNR` 指标上效果的提升空间还比较大，说明当前方法的漏报率还有待进一步降低，在后续的工作中，可以引入更多能体现代码语义的特征，比如 `API` 调用序列和程序切片的结合。另外多个实验之间的评估指标差距不是很大，说明需要进一步提取出对漏洞更具有代表性的特征以提高检测表现。

(2) 针对特征的表示学习方法，可以使用 `BERT`，本文仅使用 `word2vec`，比较单一，另外还可以考虑使用深度学习方法，将深度学习网络的中间层获得的向量作为特征的表示向量。

参考文献

- [1] apageorgiou C P, Oren M, Poggio T. A general framework for object detection[C]//Sixth International Conference on Computer Vision (IEEE Cat. No. 98CH36271). IEEE, 1998: 555-562.
- [2] Fan G , Diao X , Yu H , et al. Software Defect Prediction via Attention-Based Recurrent Neural Network[J]. Scientific Programming, 2019, 2019:1-14.
- [3] Li Z, Zou D, Xu S, et al. Vuldeepecker: A deep learning-based system for vulnerability detection[J]. arXiv preprint arXiv:1801.01681, 2018.
- [4] Li Y, Ma L, Shen L, et al. Open source software security vulnerability detection based on dynamic behavior features[J]. PloS one, 2019, 14(8).
- [5] <https://cwe.mitre.org/>
- [6] <https://nvd.nist.gov/>
- [7] <http://www.cnnvd.org.cn/>
- [8] Li Z, Zou D, Xu S, et al. Sysevr: A framework for using deep learning to detect software vulnerabilities[J]. arXiv preprint arXiv:1807.06756, 2018.
- [9] <https://www.checkmarx.com/>
- [10] Mikolov T , Chen K , Corrado G , et al. Efficient Estimation of Word Representations in Vector Space[J]. Computer ence, 2013.
- [11] Gu X, Zhang H, Kim S. Deep code search[C]//2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE). IEEE, 2018: 933-944.
- [12] Russell R, Kim L, Hamilton L, et al. Automated vulnerability detection in source code using deep representation learning[C]//2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA). IEEE, 2018: 757-762.

- [13] Sampaio L, Garcia A. Exploring context-sensitive data flow analysis for early vulnerability detection[J]. Journal of Systems and Software, 2016, 113: 337-361.
- [14] Shin Y, Meneely A, Williams L, et al. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities[J]. IEEE transactions on software engineering, 2010, 37(6): 772-787.
- Cubero-Fernandez A, Rodriguez-Lozano F J, Villatoro R, et al Efficient pavement crack detection and classification[J]. EURASIP Journal on Image and Video Processing, 2017, 2017(1): 39.
- [15] Nguyen V H, Tran L M S. Predicting vulnerable software components with dependency graphs[C]//Proceedings of the 6th International Workshop on Security Measurements and Metrics. 2010: 1-8.
- [16] Scandariato R, Walden J, Hovsepyan A, et al. Predicting vulnerable software components via text mining[J]. IEEE Transactions on Software Engineering, 2014, 40(10): 993-1006.
- [17] Huang G , Liu Z , Laurens V D M , et al. Densely Connected Convolutional Networks[J]. 2016.
- [18] <https://joern.readthedocs.io/en/latest/>
- [19] Ferrante J, Ottenstein K J, Warren J D. The program dependence graph and its use in optimization[J]. ACM Transactions on Programming Languages and Systems (TOPLAS), 1987, 9(3): 319-349.
- [20] Ottenstein K J, Ottenstein L M. The program dependence graph in a software development environment[J]. ACM Sigplan Notices, 1984, 19(5): 177-184.
- [21] Shivers O. Control-flow analysis of higher-order languages[D]. PhD thesis, Carnegie Mellon University, 1991.

- [22] Weiser M. Program slicing[J]. IEEE Transactions on software engineering, 1984 (4): 352-357.
- [23] Tip F. Program slicing [M]. Amsterdam: Centrum voor Wiskunde en Informatica, 1994.
- [24] index.php.Silva, Josep. A vocabulary of program slicing-based techniques[J]. ACM Computing Surveys, 2012, 44(3):1-41.
- [25] Sinha S, Harrold M J, Rothermel G. System-dependence-graph-based slicing of programs with arbitrary interprocedural control flow[C]//Proceedings of the 1999 International Conference on Software Engineering (IEEE Cat. No. 99CB37002). IEEE, 1999: 432-441.
- [26] Bojanowski P , Grave E , Joulin A , et al. Enriching Word Vectors with Subword Information[J]. Transactions of the Association for Computational Linguistics, 2017, 5:135-146. Joulin, Armand and Grave, Edouard and Bojanowski, Piotr and Mikolov, Tomas. "Bag of Tricks for Efficient Text Classification".
- [27] Joulin A , Grave E , Bojanowski P , et al. FastText.zip: Compressing text classification models[J]. 2016. Pennington J, Socher R, Manning C D. Glove: Global vectors for word representation[C]//Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). 2014: 1532-1543.
- [28] Support vector machines: theory and applications[M]. Springer Science & Business Media, 2005.
- [29] Liaw A, Wiener M. Classification and regression by randomForest[J]. R news, 2002, 2(3): 18-22.
- [30] Polikar R. Ensemble learning[M]//Ensemble machine learning. Springer, Boston, MA, 2012: 1-34.

- [31] Ke G, Meng Q, Finley T, et al. Lightgbm: A highly efficient gradient boosting decision tree[C]//Advances in neural information processing systems. 2017: 3146-3154.
- [32] Friedman J H. Greedy function approximation: a gradient boosting machine[J]. Annals of statistics, 2001: 1189-1232.
- [33] Yin W, Schütze H, Xiang B, et al. Abcnn: Attention-based convolutional neural network for modeling sentence pairs[J]. Transactions of the Association for Computational Linguistics, 2016, 4: 259-272.
- [34] Burt P J. Attention mechanisms for vision in a dynamic world[C]//[1988 Proceedings] 9th International Conference on Pattern Recognition. IEEE, 1988: 977-987.
- [35] Salah A A, Alpaydin E, Akarun L. A selective attention-based method for visual pattern recognition with application to handwritten digit recognition and face recognition[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2002, 24(3): 420-425.
- [36] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Advances in neural information processing systems. 2017: 5998-6008.
- [37] Kim Y. Convolutional Neural Networks for Sentence Classification[C]. empirical methods in natural language processing, 2014: 1746-1751. He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition[C]. computer vision and pattern recognition, 2016: 770-778. R. K. Srivastava, K. Greff, and J. Schmidhuber. Training very deep networks. In NIPS, 2015.
- [38] He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition[C]. computer vision and pattern recognition, 2016: 770-778.

- [39] Dam H K, Tran T, Pham T T M, et al. Automatic feature learning for predicting vulnerable software components[J]. IEEE Transactions on Software Engineering, 2018.
- [40] Software Assurance Reference Dataset, <https://samate.nist.gov/SRD/>
- [41] <https://rare-technologies.com/making-sense-of-word2vec/>
- [42] <https://github.com/c2nes/javalang>

致 谢

研究生两年时光太短暂，努力的事不要总说以后，来日并不方长，后会也可能无期，转眼间毕业论文也接近了尾声，回顾研究生生涯以及毕业论文课题的研究，期间通过同学和苏小红老师的悉心指导帮助，使我的研究课题进展顺利。

写完毕业论文，我发现，现有的成果离不开老师的帮助，离不开爸爸妈妈提供的舒适的学习环境，离不开同学的陪伴与鼓励，在这里，我真心感谢这段时间帮助过我的老师、同学和家长，对你们表示衷心的感谢。

首先我要感谢我的指导老师苏小红老师，此次毕业论文的研究课题中，由苏老师担任我的指导老师，苏老师给我提出了很多重要的意见，这对我往后工作的推进起了重要作用。在开题答辩期间，苏老师耐心修改我的开题报告，苏老师担任教授一职，平日里工作忙碌，但也不忘对我的督促和耐心讲解，经过我的努力，顺利完成了开题答辩。我更有信心去开展后续的工作。苏老师学术专业、知识渊博、工作严谨，在日常工作中，总会以每日任务来督促我们，我在苏老师身上学会了很多，学术上学习新的技术，工作上严谨、认真的态度，使我受益匪浅。

我还要感谢在校期间为我授过课的其他老师，是你们的讲解充实了我的头脑，才能在本论文中得心应手，使我灵活运用理论知识，在实践中感受真知，平时的专业课结合毕业论文完美的体现了理论与实践相结合，对我以后工作起到了促进作用。

其次还要感谢我的同学，同窗好友，一起经历风风雨雨，一起努力，一起学习，特别感谢蒋远学长，在关键时刻帮助我，毕业论文期间，遇到难以解决的问题，蒋远学长都会耐心为我讲解，给我信心。有句话讲“三人行必有我师焉”，同学身上有我值得学习的优点，在成功的路上，互相支持，度过了短暂而又美好的校园生活。

感谢在此次毕业设计中帮助过我的所有人，理论与实践相结合，为研究生生活画上了圆满的句号。